Ilton Pfleger Júnior

Virtualização de Recursos Heterogenios Para Sistemas Críticos Embarcados

Florianópolis
2027

Ilton Pfleger Júnior

Virtualização de Recursos Heterogenios Para Sistemas Críticos Embarcados

Universidade Federal de Santa Catarina

Orientador: Antônio Augusto Fröhlich

Florianópolis
2027

LISTA DE ILUSTRAÇÕES

Figura 1 – Fluxograma de Interceptação de Instruções Privilegiadas 14
Figura 2 – Hipervisor Tipo 1. 16
Figura 3 – Hipervisor Tipo 2. 16
Figura 4 – Arquitetura de Privilégios RISC-V Sem Modo Supervisor 18
Figura 5 – Arquitetura de Privilégios RISC-V Com Modo Supervisor 18
Figura 6 – Arquitetura de Privilégios RISC-V Com Extensão H 19
Figura 7 – Tradução de Endereços em Dois Estágios da Extensão H 23

Figura 8 – Diagrama de Blocos da Arquitetura do EPOS 24

Figura 9 – Diagrama de Blocos das Classes do Projeto 27
Figura 10 – Diagrama de Classes Simplificado de VMs 28
Figura 11 – Diagrama de Mapeamento de CPUs Virtuais Para Threads 28
Figura 12 – Diagrama do Fluxo de Comunicação: Convidado para Hardware 29
Figura 13 – Diagrama do Fluxo de Comunicação: Hardware para Convidado 30
Figura 14 – Diagrama de Mapeamento de Componentes Virtuais para Reais 31
Figura 15 – Diagrama de Classes Relacionadas ao Processo de Virtualização 32

Figura 16 – Emulação de Dispositivos via *Load Access Fault* 35
Figura 17 – Emulação de Dispositivos via *Store Access Fault* 36
Figura 18 – Diagrama de Virtualização de Interrupções Externas na RISC-V 37
Figura 19 – Diagrama de Envio de uma IPI Virtual na RISC-V 38

Figura 20 – Avaliação Experimental do *Overhead* de Virtualização com a Suíte MiBench. 41
Figura 21 – Distribuição de Densidade dos Tempos de Execução das Tarefas do Cenário Veicular. 42
Figura 22 – Distribuição de Densidade dos Tempos de Execução das Tarefas do Cenário Veicular com Particionamento da Cache. 43

SUMÁRIO

1 RESUMO 5

2 INTRODUÇÃO 6
2.1 Objetivos 7
2.1.1 Objetivos Gerais 7
2.1.2 Objetivos Específicos 7

3 FUNDAMENTAÇÃO TEÓRICA 8
3.1 Sistemas Embarcados 8
3.2 Sistemas Operacionais 8
3.2.1 Gerenciamento de Processos e Threads 8
3.2.2 Gerenciamento de Memória 9
3.2.3 Gerenciamento de Dispositivos de Entrada e Saída 9
3.2.4 Interrupções e Exceções 10
3.2.5 Proteção e Isolamento 11
3.3 Sistemas De Tempo Real 11
3.3.1 Classificação De Sistemas De Tempo Real 12
3.3.2 Outros Aspectos De Previsibilidade Temporal 12
3.4 Virtualização 12
3.5 Interceptação e Emulação 14
3.6 VirtIO 14
3.7 Hipervisores 15
3.7.1 Tipo 1 — Bare Metal 15
3.7.2 Tipo 2 16
3.8 RISC-V 16
3.8.1 Extensões Básicas 17
3.8.2 Níveis de Privilégio 17
3.8.3 Registradores de Controle 19
3.8.4 CLINT 20
3.8.5 PLIC 21
3.8.6 Gerenciamento de Memória Virtual 21
3.8.7 Proteção de Memória Física 21
3.8.8 Extensão H 22
3.8.8.1 Tradução de Endereços em Dois Estágios 22

4 TRABALHOS CORRELATOS . . . 24
4.1 EPOS . . . 24
4.2 Bao Hypervisor . . . 25
4.3 Miralis . . . 25
4.4 HSP-V . . . 25

5 PROJETO . . . 27
5.1 VMs . . . 28
5.2 CPU Virtual . . . 28
5.3 Dispositivos Virtuais . . . 29
5.3.1 Comunicação: Convidado para Hardware . . . 29
5.3.2 Comunicação: Hardware para Convidado . . . 30
5.4 Integração . . . 30

6 IMPLEMENTAÇÃO . . . 33
6.1 Virtualização Baseada em Modo Machine . . . 33
6.2 Virtualização de Temporizador . . . 33
6.2.1 Leitura do Temporizador . . . 34
6.2.2 Programação de Temporizador . . . 34
6.3 Virtualização de Ociosidade . . . 34
6.4 Virtualização De Dispositivos . . . 34
6.5 Virtualização de Interrupções Externas . . . 36
6.6 Virtualização da Comunicação Entre Processadores . . . 37
6.7 Atualização de Estado das CPUs Virtuais . . . 38
6.8 Escalonamento de CPUs Virtuais . . . 38

7 RESULTADOS . . . 40
7.1 Materiais . . . 40
7.2 Avaliação de Desempenho: Nativo vs. Virtualizado . . . 40
7.3 Avaliação de Determinismo e Isolamento . . . 42

8 CONCLUSÕES . . . 45

REFERÊNCIAS . . . 47

1 RESUMO

Este trabalho apresenta um hipervisor Tipo-1 leve e de tempo real em C++ para a arquitetura RISC-V, projetado para sistemas embarcados de criticidade mista sem exigir suporte nativo da extensão H de virtualização. Executando em modo Machine, a solução garante o isolamento temporal e espacial de CPU, memória e E/S por meio do mapeamento de vCPUs e da emulação de dispositivos, temporizadores e interrupções, aproveitando metaprogramação em C++ para minimizar o overhead. Em testes no SoC StarFive JH7110, o hipervisor obteve um overhead de processamento inferior a 2% com a suíte MiBench.

Palavras-chave: Virtualização; Sistemas Embarcados de Tempo Real; RISC-V; Hipervisor; Sistemas de Criticalidade Mista.

2 INTRODUÇÃO

Sistemas embarcados estão presentes em uma ampla gama de aplicações, que vão desde dispositivos médicos e sistemas de automação industrial até plataformas automotivas e missões espaciais. Diferentemente da computação de propósito geral, esses sistemas operam sob rigorosos requisitos temporais, nos quais a corretude do comportamento não depende apenas da precisão lógica dos resultados produzidos, mas também do momento exato em que esses resultados são disponibilizados. Em muitos casos, atrasos mínimos ou falhas temporais podem comprometer a segurança, a confiabilidade e até mesmo a integridade física de pessoas e equipamentos.

Tradicionalmente, a garantia desse determinismo esteve associada ao uso de plataformas dedicadas, nas quais Sistemas Operacionais de Tempo Real (RTOSs) são empregados para assegurar previsibilidade temporal, isolamento entre tarefas e cumprimento de restrições rígidas de escalonamento. Entretanto, a evolução do software contemporâneo deslocou grande parte dos ecossistemas de desenvolvimento — bibliotecas, frameworks, ferramentas e serviços — para ambientes baseados em Sistemas Operacionais de Propósito Geral (GPOSs) - especialmente Linux. Embora esses sistemas ofereçam grande riqueza funcional e amplo suporte da comunidade, sua adoção em contextos críticos embarcados demanda esforços adicionais de adaptação, validação e integração.

Como resultado, cresce a necessidade de consolidar, em uma mesma plataforma computacional, cargas de trabalho com requisitos distintos e frequentemente conflitantes: de um lado, aplicações críticas, sensíveis a prazos temporais e dependentes de comportamento determinístico; de outro, aplicações complexas, orientadas a funcionalidades avançadas e dependentes de ambientes generalistas.

Nesse cenário, técnicas de virtualização surgem como uma alternativa promissora, pois permitem a coexistência de múltiplos ambientes de execução sobre o mesmo hardware. Dessa forma, torna-se possível executar aplicações críticas e não críticas de maneira controlada, reduzindo custos de integração e aumentando o aproveitamento da plataforma.

Paralelamente, a arquitetura aberta RISC-V destaca-se por sua flexibilidade, modularidade e ausência de barreiras proprietárias, impulsionando o desenvolvimento de soluções customizadas e de baixo custo. Contudo, esses processadores mais simples raramente dispõem de extensões de hardware dedicadas à virtualização. Esse cenário gera um gargalo relevante, visto que a maioria das plataformas de virtualização atuais depende estritamente desses recursos físicos para funcionar.

Por isso, este trabalho propõe o projeto e a implementação, em C++, de uma infraestrutura de virtualização voltada a sistemas embarcados baseados em RISC-V. A proposta tem como foco a previsibilidade temporal, o isolamento entre cargas de trabalho,

o reduzido custo de execução e a possibilidade de utilização em plataformas de recursos computacionais limitados. Busca-se, dessa forma, contribuir para o desenvolvimento de plataformas embarcadas capazes de executar, de forma simultânea e isolada, cargas de trabalho heterogêneas, conciliando requisitos de desempenho, determinismo temporal e confiabilidade.

2.1 OBJETIVOS

2.1.1 Objetivos Gerais

Desenvolver e avaliar uma infraestrutura de virtualização em C++ para sistemas embarcados baseados na arquitetura RISC-V, focada no particionamento estático de recursos, no isolamento de cargas de trabalho de criticidade mista e na garantia da previsibilidade temporal para aplicações de tempo real.

2.1.2 Objetivos Específicos

- Projetar e implementar em software o núcleo do hipervisor para o gerenciamento, criação, inicialização e execução de máquinas virtuais em plataformas RISC-V;
- Implementar o particionamento estático de processadores, memória e dispositivos de Entrada e Saída (IO), garantindo o isolamento espacial e temporal entre domínios;
- Validar a infraestrutura funcionalmente por meio da execução de Sistemas Operacionais (SOs) convidados representativos;
- Quantificar o impacto do hipervisor no desempenho do sistema, mensurando *overheads* computacionais e a previsibilidade temporal sob restrições de tempo real.

3 FUNDAMENTAÇÃO TEÓRICA

3.1 SISTEMAS EMBARCADOS

Sistemas embarcados são soluções computacionais projetadas para executar funções dedicadas no interior de equipamentos maiores. Amplamente utilizados em setores como automação industrial, indústria automotiva, medicina, telecomunicações e *Internet das Coisas* (IoT), esses sistemas destacam-se pela forte integração entre hardware e software. Como operam frequentemente sob restrições severas de processamento, memória, armazenamento e consumo de energia, exigem um desenvolvimento altamente otimizado para entregar máxima eficiência com baixo custo computacional.

3.2 SISTEMAS OPERACIONAIS

Um SO é um software responsável por gerenciar os recursos de hardware de um sistema computacional e fornecer serviços para a execução de aplicações. Atuando como uma camada de abstração entre o hardware e os programas, o SO simplifica o desenvolvimento de software ao oferecer mecanismos padronizados para acessar o processador, memória, dispositivos de IO, dispositivos armazenamento de dados, etc. A evolução dos sistemas operacionais permitiu o surgimento de plataformas cada vez mais complexas, capazes de executar múltiplas aplicações simultaneamente e fornecer diversos serviços ao usuário.

Nas subseções seguintes, são apresentados os principais mecanismos empregados pelos sistemas operacionais:

3.2.1 Gerenciamento de Processos e Threads

Um processo é a instância de um programa em execução, responsável por reservar os recursos do sistema e disponibilizar o ambiente de processamento. Dentro dele, as threads atuam como unidades independentes de execução, cada uma mantendo seu próprio contexto exclusivo — composto por estado, registradores e pilha. Ao abrigar múltiplas threads, o processo habilita a execução concorrente de tarefas. Em CPUs multinúcleo, o sistema distribui essas threads entre os núcleos físicos, alcançando o paralelismo real: a execução simultânea de instruções para maximizar o desempenho.

O gerenciamento da execução dessas threads é atribuído ao escalonador do sistema operacional. Essa unidade é responsável por coordenar a distribuição do tempo de CPU e alternar a execução entre as threads por meio da troca de contexto — processo no qual o

estado atual dos registradores de uma thread é salvo para que outra possa ser executada sem perda de dados.

Essa orquestração contínua dita a movimentação da thread ao longo de seu ciclo de vida, composto costumeiramente pelos estados: pronta, executando, esperando e finalizada. No estado pronta, a thread aguarda na fila do escalonador a sua vez de assumir o processador. Ao obter a posse da CPU, ela passa para o estado de execução. Quando necessita aguardar um evento externo, a thread transita para o estado de espera até que possa retornar à fila de prontos. Esse bloqueio é fundamental para a eficiência energética do sistema, pois, caso todas as threads estejam ociosas ou bloqueadas, a plataforma pode entrar em modo de baixo consumo — idle. Por fim, a thread finalizada encerra seu fluxo e aguarda a desalocação definitiva de seus recursos pelo sistema operacional.

3.2.2 Gerenciamento de Memória

Aplicações, de modo geral, demandam diferentes tamanhos e tipos de memória. No entanto, elas não podem acessar livremente todo o espaço disponível na plataforma, pois isso comprometeria a estabilidade do sistema. Por isso, os SOs ficam responsáveis por alocar porções específicas de memória para cada aplicação e privá-las de todo o resto. Dessa forma, se uma aplicação tentar acessar uma região que não lhe pertence, o sistema gerará uma exceção que resultará no seu encerramento. Caso a memória previamente alocada se torne insuficiente, a aplicação pode solicitar porções adicionais ao SO.

A memória previamente alocada para operações gerais e alocações de objetos locais é chamada de pilha — *stack*. Todavia, toda a memória extra alocada é chamada de memória dinâmica — *heap*.

Para proteger as porções de memória que não pertencem a um determinado processo, o SO precisa de algum mecanismo de Memory Management Unit (MMU). MMUs primitivas eram capazes apenas de proteger porções estáticas da memória. Porém, com a evolução dos sistemas computacionais, esse mecanismo foi melhorado e hoje emprega a responsabilidade de separar memória física da memória virtual — um conceito que consiste em um espaço de endereçamento abstraído e independente para cada processo, permitindo que o SO gerencie e traduza esses endereços virtuais em posições reais dos dispositivos de hardware de forma dinâmica e segura.

3.2.3 Gerenciamento de Dispositivos de Entrada e Saída

Dispositivos de IO constituem mecanismos que viabilizam a troca de dados e sinais de controle entre o sistema computacional e o meio externo. O controle e a operação desses dispositivos são intermediados por *device drivers*, componentes do software básico responsáveis por implementar a comunicação e o gerenciamento específico de cada dispositivo.

Durante seu ciclo de execução, praticamente todas as aplicações necessitam, em algum momento, interagir com dispositivos de IO, seja para exibir informações em uma interface serial ou em um monitor de vídeo, seja para armazenar ou recuperar dados. Entretanto, diferentes plataformas podem possuir dispositivos distintos, com características e interfaces de comunicação próprias. Dessa forma, as aplicações não são responsáveis por implementar diretamente os mecanismos de comunicação com o hardware. Essa responsabilidade é atribuída ao SO, que fornece uma camada de abstração capaz de ocultar as particularidades dos dispositivos e oferecer uma interface uniforme às aplicações.

Além de fornecer essa abstração, o SO atua como mediador no acesso aos dispositivos, coordenando seu compartilhamento entre as diferentes aplicações. Esse gerenciamento permite controlar o acesso concorrente aos recursos de IO, evitando conflitos e garantindo que sua utilização ocorra de forma segura e consistente.

3.2.4 Interrupções e Exceções

As interrupções constituem-se em um mecanismo fundamental de controle em arquiteturas computacionais, permitindo que o processador suspenda temporariamente a execução de uma tarefa para atender a eventos de alta prioridade. Este paradigma é essencial para a operação assíncrona de sistemas modernos, viabilizando a reação imediata a sinais de hardware, periféricos ou condições de erro.

Quando uma interrupção é disparada, o fluxo de execução é desviado para uma rotina especializada denominada *Interrupt Service Routine* (ISR). O tratamento ocorre geralmente em três etapas:

1. Salvamento de Contexto: o estado atual da CPU é salvo na pilha ou em uma estrutura de dados específica;

2. Execução da ISR: o processador executa o código responsável por tratar o evento que causou a interrupção;

3. Restauração do Contexto: o estado anterior é restaurado, permitindo que a tarefa originalmente interrompida retome seu processamento a partir do ponto em que foi suspensa;

Nessa mesma lógica, exceções são mecanismos de controle semelhantes às interrupções, pois ambos permitem que o processador suspenda temporariamente o fluxo normal de execução para tratar eventos específicos. No entanto, enquanto as interrupções são geralmente causadas por eventos assíncronos externos à CPU — como sinais de periféricos ou temporizadores — as exceções são geradas de forma síncrona pelo processador durante a execução de uma instrução. Este mecanismo é fundamental para o tratamento de

condições anômalas, tais como divisões por zero, acessos inválidos à memória, estouro de pilha ou execução de instruções não suportadas.

Ao detectar uma exceção, o processador desvia o fluxo para uma rotina de tratamento — geralmente a mesma das interrupções. A principal diferença operacional reside no pós-processamento: diferentemente das interrupções, que usualmente retomam o fluxo da tarefa original, o SO pode decidir encerrar a aplicação que causou a exceção caso ela seja classificada como fatal.

3.2.5 Proteção e Isolamento

Além de fornecer suas abstrações, um SO deve garantir a proteção do próprio sistema contra processos defeituosos e o isolamento entre diferentes aplicações. Em sistemas críticos, essas propriedades são fundamentais para evitar falhas em cascata e assegurar a previsibilidade e a confiabilidade da execução. Essa separação entre o núcleo do sistema e as aplicações fundamenta-se em mecanismos de hardware, destacando-se os níveis de privilégio de execução — que restringem operações sensíveis apenas a códigos autorizados — e a proteção de memória, que delimita as regiões de memória acessíveis por cada processo. Dessa forma, o SO opera em um nível mais privilegiado, mantendo as aplicações contidas e restritas aos seus próprios ambientes estabelecendo uma separação entre o núcleo do sistema e os processos que ele executa.

3.3 SISTEMAS DE TEMPO REAL

Sistemas que operam sob restrições de tempo real são sistemas nos quais a reatividade temporal é extremamente importante para a corretude ou até mesmo para a segurança do ambiente de execução. Nesses sistemas, não é suficiente que uma operação produza um resultado correto ou que seja extremamente rápida, sendo também necessário que esse resultado seja produzido dentro de um intervalo de tempo determinado (STANKOVIC, 1988).

Para atender a esses requisitos, são empregados RTOSs, projetados para fornecer mecanismos de gerenciamento de tarefas e recursos que favoreçam a previsibilidade e o cumprimento de restrições temporais. Diferentemente de GPOSs, nos quais o objetivo principal geralmente está relacionado ao aumento do desempenho médio, da utilização dos recursos ou da responsividade percebida pelo usuário, um RTOS prioriza a capacidade de determinar e controlar os tempos envolvidos na execução das tarefas.

Os conceitos centrais associados aos RTOSs são o *deadline* e o determinismo. O *deadline* - ou prazo limite - refere-se ao instante máximo em que a execução de uma determinada tarefa deve ser concluída, cabendo ao RTOS gerenciar os recursos para assegurar o cumprimento desta restrição. Por sua vez, o determinismo — frequentemente associ-

ado à previsibilidade do sistema — garante que o comportamento temporal do software seja conhecido e limitado, com a determinação do tempo de execução no pior caso ou *Worst-Case Execution Time* (WCET).

O elemento central do núcleo de um RTOS que viabiliza essa previsibilidade é o algoritmo de escalonamento de tarefas - adaptado para tempo real. Entre os algoritmos mais consolidados na literatura, destacam-se o *Earliest Deadline First* (EDF), utilizado em cenários com prioridades dinâmicas — nos quais o conjunto de tarefas ou seus prazos podem variar em tempo de execução —, e o *Rate-Monotonic* (RM), voltado a cenários estáticos, onde as tarefas e seus períodos de ativação são totalmente conhecidos em fase de projeto.

3.3.1 Classificação De Sistemas De Tempo Real

Tais sistemas são classificados de acordo com as consequências do não cumprimento dos requisitos temporais definidos em sua elaboração. Sistemas nos quais a perda de um prazo pode causar danos catastróficos, como a perda de vidas humanas ou a destruição de equipamentos de alto valor, são classificados como Hard Real-Time. Por outro lado, sistemas nos quais o atraso temporal não é desejável, mas também não é fatal ou crítico — como o atraso na entrega de quadros em serviços de vídeo —, são chamados de Soft Real-Time.

3.3.2 Outros Aspectos De Previsibilidade Temporal

- **Latência:** intervalo de tempo entre a ocorrência de um evento e o início da ação correspondente do sistema.
- ***Jitter*:** variação temporal observada na latência ou no instante de ocorrência de eventos em relação ao comportamento esperado.
- ***Overhead*:** custo computacional adicional introduzido por algum mecanismo.

3.4 VIRTUALIZAÇÃO

A virtualização é uma técnica computacional que abstrai recursos físicos — como capacidade de processamento, memória, armazenamento e infraestrutura de rede —, separando a camada de hardware das aplicações e ambientes de execução. Por meio do gerenciamento por uma camada intermediária, ela permite criar instâncias virtuais isoladas e independentes que compartilham a mesma infraestrutura física de forma transparente, otimizando o uso de recursos e aumentando a flexibilidade do sistema.

No âmbito da virtualização de sistemas e hardware, destacam-se duas abordagens principais para o gerenciamento de execução: a virtualização completa e a paravirtualização.

Na virtualização completa ou *full virtualization*, o sistema convidado executa sem ter conhecimento explícito de que reside em um ambiente virtualizado. Nesse cenário, instruções privilegiadas são interceptadas pelo virtualizador por meio de exceções de hardware. Embora essa abordagem garanta total compatibilidade sem a necessidade de modificar o sistema convidado, a constante alternância e o salvamento de contexto associados às exceções podem impactar o desempenho geral.

Em contrapartida, a paravirtualização estabelece uma relação cooperativa entre o sistema convidado e a camada de virtualização. O sistema convidado é modificado para reconhecer a presença do virtualizador e, em vez de executar instruções privilegiadas diretamente, realiza chamadas explícitas: reduzindo o custo de processamento e aumentando a eficiência das operações.

Independentemente da abordagem adotada, a base de um virtualizador de hardware fundamenta-se no mapeamento de recursos. Segundo (GOLDBERG, 1973b), essa dinâmica é formalizada por uma função $f : V \to R \cup \{\Omega\}$, responsável por traduzir elementos do domínio virtual — V — para o domínio físico — R, retornando Ω quando um recurso não está mapeado. Para operacionalizar essa abstração, o virtualizador deve possuir quatro características essenciais:

1. **Banco de dados para armazenar f:** Estrutura onde o virtualizador mantém o estado das correspondências entre os recursos virtuais e físicos.

2. **Mecanismo para invocar f:** Componente de hardware ou software encarregado de consultar e disparar a função f quando necessário.

3. **Mecânica da composição de mapeamentos:** Lógica que combina a tradução interna do sistema convidado — ϕ — com a do virtualizador — f.

4. **Ação diante de uma interceptação:** Procedimento acionado quando $f(v) = \Omega$ ou ao interceptar instruções privilegiadas.

Em suma, a virtualização consolida-se como um paradigma que desvincula os programas do hardware físico subjacente. Ao atuar de forma transparente como uma camada intermediária, o virtualizador orquestra essa comunicação e gerencia livremente os recursos, viabilizando o compartilhamento seguro e eficiente de um mesmo dispositivo físico entre múltiplos ambientes de execução.

3.5 INTERCEPTAÇÃO E EMULAÇÃO

Do inglês *trap-and-emulate*, interceptação e emulação trata-se de um método utilizado para viabilizar a virtualização em sistemas nos quais o suporte de hardware não é completamente implementado. Nesse modelo, quando a aplicação executada em ambiente virtualizado tenta realizar uma operação que depende de recursos privilegiados, essa operação é interceptada e tratada como uma exceção. O controle é então transferido para um nível mais privilégiado, geralmente o hipervisor, que fica responsável por emular em software o comportamento esperado da instrução ou operação originalmente solicitada.

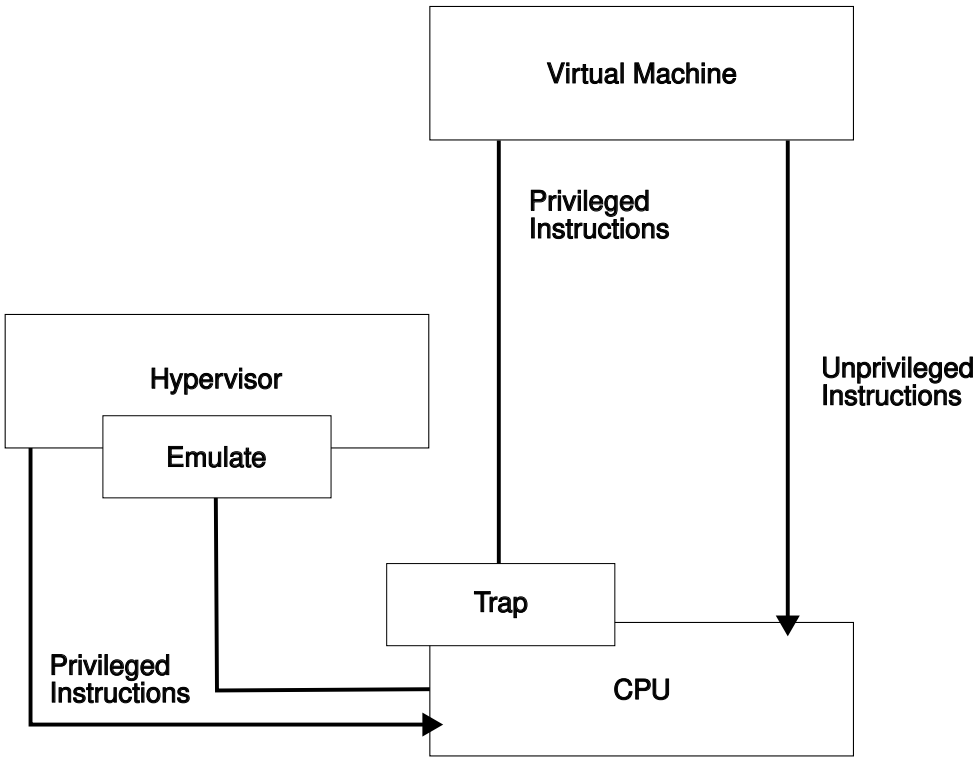


Figura 1 – Fluxograma de Interceptação de Instruções Privilegiadas

3.6 *VIRTIO*

VirtIO (RUSSELL, 2008) é um padrão para drivers de dispositivos paravirtualizados, projetado para unificar e otimizar a interface de comunicação entre os sistemas convidados e o hipervisor. Em vez de emular hardwares legados complexos e específicos, o hipervisor expõe uma série de dispositivos virtuais padronizados, permitindo que o SO convidado acesse recursos de rede, armazenamento e console de maneira eficiente e com baixa sobrecarga.

Para viabilizar essa comunicação, o modelo utiliza as chamadas *Virtqueues* — filas circulares compartilhadas responsáveis pelo envio e recebimento das requisições. Dependendo das operações suportadas, cada dispositivo virtual pode alocar uma ou mais dessas

filas. Internamente, uma *Virtqueue* divide-se em três componentes principais: uma tabela de descritores (*Descriptor Table*), um anel de índices disponíveis (*Available Ring*) e um anel de índices já utilizados (*Used Ring*).

Na prática, o ciclo de vida de uma operação de IO ocorre em duas etapas complementares. Inicialmente, quando o sistema convidado deseja realizar uma operação, o dispositivo *VirtIO* escreve os dados na memória compartilhada, insere o descritor correspondente no *Available Ring* e notifica o hipervisor sobre a nova pendência.

Em resposta, o hipervisor consome os descritores desse anel, processa a operação solicitada e registra o resultado no *Used Ring*. Por fim, uma interrupção é enviada ao sistema convidado para sinalizar a conclusão da tarefa. Graças a esse design, múltiplas requisições podem ser tratadas de maneira assíncrona e em lote, o que minimiza as trocas de contexto e reduz expressivamente o custo inerente à virtualização.

Desse modo, o *VirtIO* consolida-se como uma solução fundamental para a infraestrutura de virtualização moderna, pois alcança um equilíbrio entre a flexibilidade de uma interface padronizada e o desempenho da paravirtualização.

3.7 HIPERVISORES

Um hipervisor, também conhecido como Monitor de Máquina Virtual (VMM), é uma plataforma que cria e executa máquinas virtuais por meio da virtualização. Segundo (POPEK; GOLDBERG, 1974), um VMM possui três propriedades essenciais: eficiência, controle de recursos e equivalência. Quanto à eficiência, toda instrução não privilegiada deve ser executada diretamente no hardware. No controle de recursos, o VMM mantém domínio total do sistema, garantindo que as Máquinas Virtuais (VMs) não afetem a estabilidade global. Por fim, na equivalência, o ambiente da VM deve ser funcionalmente indistinguível do hardware real — salvo exceções, simulando acesso privilegiado exclusivo.

Na visão de (GOLDBERG, 1973a), fundamentalmente, existem dois tipos de hipervisores: 1 e 2.

3.7.1 Tipo 1 — Bare Metal

Hipervisores de tipo 1, também conhecidos como bare metal, são aqueles que executam diretamente sobre o hardware físico, sem a necessidade de um sistema operacional hospedeiro intermediário. Por operar diretamente sobre o hardware, esse tipo de hipervisor geralmente apresenta melhor desempenho e menor sobrecarga.

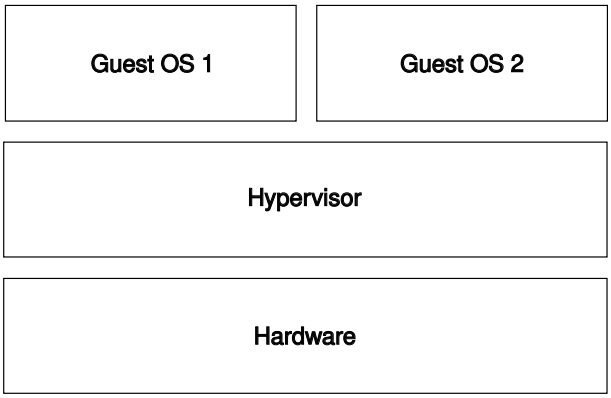


Figura 2 – Hipervisor Tipo 1.

3.7.2 Tipo 2

Diferentemente da solução bare metal, um hipervisor do tipo 2 é executado como um processo dentro de um SO hospedeiro. Porém, apesar de garantir os mesmos recursos do tipo 1, sua arquitetura impõe uma camada intermediária a mais de software, o que restringe a eficiência no acesso aos componentes físicos, gerando uma latência adicional e, consequentemente, um menor desempenho.

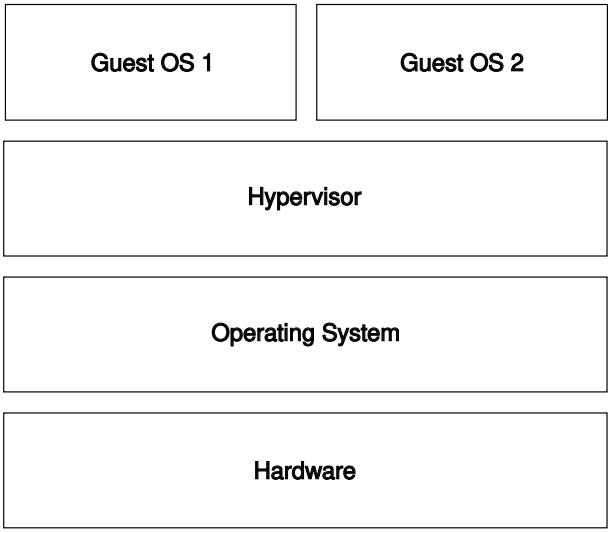


Figura 3 – Hipervisor Tipo 2.

3.8 RISC-V

A RISC-V é uma Instruction Set Architecture (ISA) baseada nos princípios Reduced Instruction Set Computer (RISC). Diferentemente de arquiteturas proprietárias, a RISC-V possui uma especificação aberta e livre de *royalties*, o que permite sua utilização tanto em contextos acadêmicos quanto em aplicações comerciais.

Essa ISA define um conjunto básico de instruções, responsável pelas funcionalidades essenciais do processador. No entanto, sua principal característica é a modularidade, permitindo que diferentes extensões sejam incorporadas conforme as necessidades da plataforma. Dessa forma, é possível adaptar a arquitetura para cenários variados, desde sistemas embarcados com restrições de recursos até sistemas de alto desempenho.

3.8.1 Extensões Básicas

I Adiciona suporte às instruções básicas de operações com números inteiros. Conforme descrito em (INTERNATIONAL, 2024), essa extensão deve estar presente em qualquer implementação RISC-V, pois define as operações fundamentais de aritmética e manipulação de registradores.

Zicsr Adiciona suporte às instruções de acesso e manipulação de Control and Status Registers (CSRs).

H Adiciona suporte ao modo H (Hipervisor).

S Adiciona suporte ao modo S (Supervisor).

M Adiciona suporte a operações de multiplicação e divisão de números inteiros.

A Adiciona suporte a operações atômicas em memória.

F Adiciona suporte a números em ponto flutuante de precisão simples (32 bits).

D Adiciona suporte a números em ponto flutuante de precisão dupla (64 bits).

C Adiciona suporte a instruções comprimidas de 16 bits.

V Adiciona suporte a operações vetoriais para *Single Instruction, Multiple Data* (SIMD).

B Adiciona suporte à manipulação de bits em hardware, otimizando operações bit a bit.

G Aglomerado geral de extensões (*General Purpose*) que inclui I, M, A, F, D e as extensões de controle Zicsr e Zifencei.

3.8.2 Níveis de Privilégio

Assim como a maior parte das arquiteturas modernas, a RISC-V utiliza um modelo hierárquico de privilégios para garantir a segurança e a estabilidade do sistema. Esses níveis determinam quais operações e recursos de hardware podem ser acessados em cada contexto de execução. Os modos implementados em uma plataforma dependem diretamente das extensões suportadas, estruturando-se tipicamente em três cenários principais:

- **Sem a extensão S:** Utiliza apenas o modo *Machine* e o modo *User*, indicado para sistemas embarcados mais simples e sem suporte a sistemas operacionais baseados em supervisor.

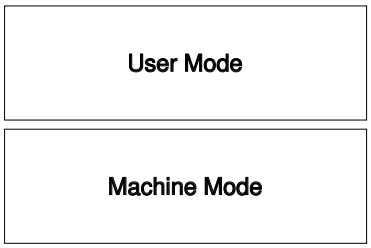


Figura 4 – Arquitetura de Privilégios RISC-V Sem Modo Supervisor

- **Com a extensão S:** Adiciona o modo *Supervisor*, permitindo a execução de sistemas operacionais completos que exigem isolamento entre o kernel e as aplicações de usuário.

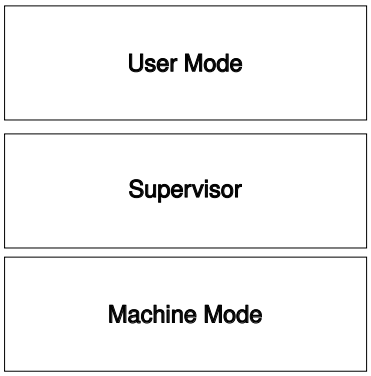


Figura 5 – Arquitetura de Privilégios RISC-V Com Modo Supervisor

- **Com a extensão H:** Introduz o suporte aos modos *Virtual User* e *Virtual Supervisor*, estendendo os modos de privilégio para permitir a execução de hipervisores modernos.

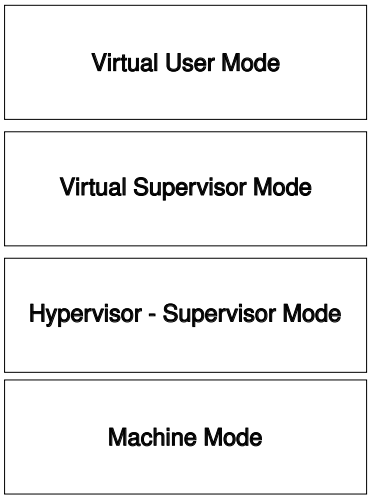


Figura 6 – Arquitetura de Privilégios RISC-V Com Extensão H

3.8.3 Registradores de Controle

Os CSRs são registradores especiais utilizados para controlar e monitorar o funcionamento do processador. Eles armazenam informações essenciais relacionadas ao estado da máquina, ao tratamento de exceções e interrupções, à configuração de privilégios e ao controle de funcionalidades específicas da arquitetura.

Esses registradores são organizados de forma segregada por nível de privilégio, sendo que cada nível dispõe do seu próprio conjunto de CSRs. Esse isolamento é fundamental para a segurança e a integridade do sistema, pois garante uma política rígida de acessibilidade: um nível de privilégio só pode acessar seus próprios registradores ou aqueles pertencentes a níveis menos privilegiados. Qualquer tentativa de acesso indevido a um CSR de um nível mais privilegiado resulta em uma exceção de instrução ilegal, impedindo que código menos privilegiado interfira diretamente nas camadas críticas do sistema.

Os principais CSRs são:

- `mhartid`: mantém o identificador único de cada núcleo da CPU;
- `mstatus` e `mstatush`: mantêm os principais bits de configuração relacionados à arquitetura;
- `mtvec`: armazena o endereço base do vetor de manipuladores de exceções e interrupções no modo *Machine*;
- `mepc`: guarda o endereço da instrução que causou a exceção ou interrupção no modo *Machine*;
- `mcause`: armazena a causa de uma instrução ou exceção no modo *Machine*;

- `mtval`: armazena dados adicionais de uma exceção no modo *Machine*;
- `mscratch`: registrador temporário de rascunho usado para salvar ponteiros de contexto durante manipuladores no modo *Machine*;
- `mideleg` e `medeleg`: configuram a delegação de interrupções e exceções do modo *Machine* para modos de menor privilégio;
- `mip`: mantém as interrupções atualmente pendentes;
- `mie`: mantém as interrupções atualmente habilitadas;
- `sstatus`: versão restrita e visível do registrador de status para o modo *Supervisor*;
- `stvec`: armazena o endereço base do manipulador de *traps* no modo *Supervisor*;
- `sepc`: guarda o endereço da instrução que causou a exceção ou interrupção no modo *Supervisor*;
- `scause`: armazena a causa da interrupção ou exceção no modo *Supervisor*;
- `stval`: guarda informações adicionais relativas à exceção ocorrida no modo *Supervisor*;
- `sscratch`: registrador de rascunho temporário do modo *Supervisor*;
- `sip`: correspondente do modo *Supervisor* ao `mip`;
- `sie`: correspondente do modo *Supervisor* ao `mie`;
- `satp`: controla a tradução de endereços virtuais e a paginação da memória no modo *Supervisor*.

3.8.4 Core Local Interrupter (CLINT)

O CLINT é o mecanismo responsável por gerenciar interrupções locais — especificamente as de software e de temporização — em plataformas RISC-V.

Para a gestão de tempo, o CLINT possui dois registradores centrais: `mtime` e `mtimecmp`. O `mtime` funciona como a referência temporal principal da arquitetura, mantendo o SO ciente da passagem do tempo. O `mtimecmp`, por sua vez, determina o momento em que uma interrupção de temporização deve ser disparada: quando o valor de `mtime` torna-se maior ou igual ao de `mtimecmp`, o CLINT sinaliza o bit de interrupção temporal no registrador *Machine Interrupt Pending* (MIP).

Em relação às interrupções de software, o CLINT disponibiliza um registrador *Machine Software Interrupt Pending* (MSIP) mapeado em memória para cada núcleo. Dessa forma, um núcleo pode disparar uma interrupção de software em outro ativando o bit zero do registrador MSIP pertencente ao núcleo de destino.

3.8.5 Platform-Level Interrupt Controller (PLIC)

O funcionamento do PLIC baseia-se na centralização e priorização de solicitações provenientes de periféricos externos. Quando uma interrupção atende aos requisitos de habilitação e ao limiar de prioridade, o PLIC notifica o núcleo ativando o bit correspondente de interrupção externa no registrador MIP, como *Machine External Interrupt Pending* (MEIP) ou *Supervisor External Interrupt Pending* (SEIP). A partir dessa notificação, para identificar e tratar a interrupção, o SO deve se comunicar ativamente com o PLIC.

3.8.6 Gerenciamento de Memória Virtual

A MMU na arquitetura RISC-V é responsável por realizar a tradução de endereços virtuais em endereços físicos e por verificar as permissões de acesso à memória. O mecanismo de tradução utilizado pelos sistemas operacionais é configurado por meio do registrador de controle *Supervisor Address Translation and Protection* (SATP), disponível no modo *Supervisor*.

Seu funcionamento baseia-se na paginação da memória, na qual os endereços virtuais são divididos em páginas, tipicamente com granularidade de 4,KiB. O SO mantém tabelas de páginas que associam essas páginas virtuais a regiões da memória física e definem as permissões de acesso, como leitura, escrita e execução. Dessa forma, quando um processo acessa um endereço virtual, a MMU realiza a tradução correspondente e verifica se a operação solicitada é permitida. Caso o endereço não esteja mapeado ou a operação viole as permissões estabelecidas, ocorre uma exceção de acesso à memória.

3.8.7 Proteção de Memória Física

Em sistemas que não possuem o modo supervisor — ou como camada adicional de segurança —, a arquitetura RISC-V emprega o mecanismo de Physical Memory Protection (PMP). Exclusivamente no modo *Machine*, o firmware configura os registradores de PMP para delimitar quais regiões da memória física podem ser acessadas pelos modos *User* e *Supervisor*, assegurando o isolamento por hardware mesmo na ausência ou antes da inicialização de uma MMU.

Para gerenciar essas regras, o mecanismo baseia-se em dois tipos de registradores: os registradores de configuração — *pmpcfg*, que definem as permissões de acesso — leitura, escrita e execução, o modo de pareamento de endereços e o bit de travamento para cada entrada; e os registradores de endereço — *pmpaddr*, que armazenam os endereços de referência ou parâmetros empregados pelo hardware para delimitar o tamanho e os limites das regiões protegidas.

O mecanismo de PMP definido pela ISA da RISC-V possui três modos ativos de pareamento de endereços:

- ***Top of Range* (TOR):** define uma faixa contínua de memória onde o registrador $pmpaddr_i$ atua como o limite superior — exclusivo — e o registrador anterior $pmpaddr_{i-1}$ atua como o limite inferior — inclusivo. Para a primeira entrada ($i = 0$), o limite inferior é considerado o endereço 0×0;
- ***Naturally Aligned 4-Byte* (NA4):** mapeia um bloco fixo de exatamente 4 bytes apontado diretamente pelo registrador *pmpaddr*;
- ***Naturally Aligned Power-of-Two* (NAPOT):** define blocos com tamanhos que são potências de 2. O tamanho e o alinhamento da região são codificados nos bits menos significativos do próprio registrador *pmpaddr*.

3.8.8 Extensão H

A extensão H introduz suporte nativo à virtualização em nível de hardware na arquitetura RISC-V. O principal objetivo dessa extensão é permitir a execução eficiente de um VMM. Para viabilizar esse isolamento e controle, a extensão H expande o modelo de privilégios da arquitetura, adicionando novos modos de execução:

- **Virtual User:** modo no qual rodam as aplicações de usuário dentro do convidado.
- **Virtual Supervisor:** modo no qual o SO convidado é executado.
- **Hypervisor:** modo supervisor estendido, onde o hipervisor opera para gerenciar os recursos das VMs.

3.8.8.1 Tradução de Endereços em Dois Estágios

Além dos novos modos de privilégio, a extensão H adiciona uma camada de tradução de memória para suportar a virtualização. Como múltiplos SOs convidados dependem da MMU da plataforma — que normalmente seria controlada apenas pelo hipervisor —, a arquitetura introduz os registradores *vsatp* e *hgatp* — substitutos e complementos ao registrador padrão *satp* —. Isso viabiliza uma tradução de endereços em dois estágios: o *vsatp* mapeia os endereços virtuais do convidado para endereços físicos intermediários, enquanto o *hgatp* traduz esses endereços intermediários para a memória física real da máquina.

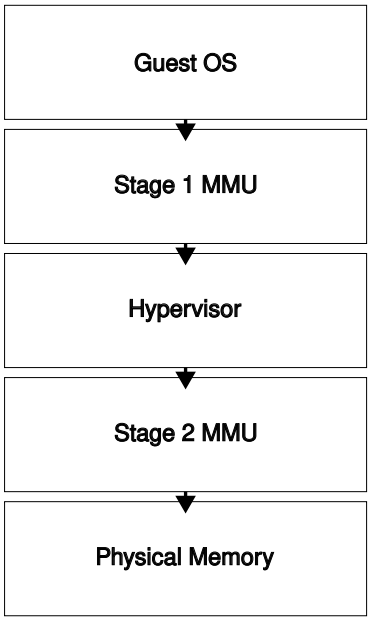


Figura 7 – Tradução de Endereços em Dois Estágios da Extensão H

4 TRABALHOS CORRELATOS

4.1 EPOS

O Embedded Parallel Operating System (EPOS) (FRöHLICH; SCHRöEDER-PREIKSCHAT, 1999) é um SO de tempo real orientado a objetos, desenvolvido em C++ e voltado para sistemas embarcados e aplicações dedicadas. Seu desenvolvimento fundamenta-se na metodologia *Application-Driven Embedded System Design* (ADESD), que busca automatizar a construção de sistemas a partir dos requisitos específicos de cada aplicação. Essa abordagem permite especializar o sistema para cenários concretos de execução, incorporando apenas os componentes estritamente necessários e eliminando *overheads* e consumo desnecessários de recursos.

Uma característica central do EPOS é o emprego intensivo de metaprogramação e abstrações em C++ para deslocar decisões de configuração e especialização para o tempo de compilação. Essa estratégia possibilita a criação de componentes altamente especializados sem impor custos em tempo de execução — propriedade indispensável em sistemas embarcados com restrições severas de recursos.

Como ilustrado na Figura 8, a arquitetura do EPOS é organizada em modelos de classes C++, estruturadas em diferentes níveis de abstração e de responsabilidade. Nessa hierarquia, quanto mais alto for o nível de uma classe, mais genérica e menos especializada ela se apresenta.

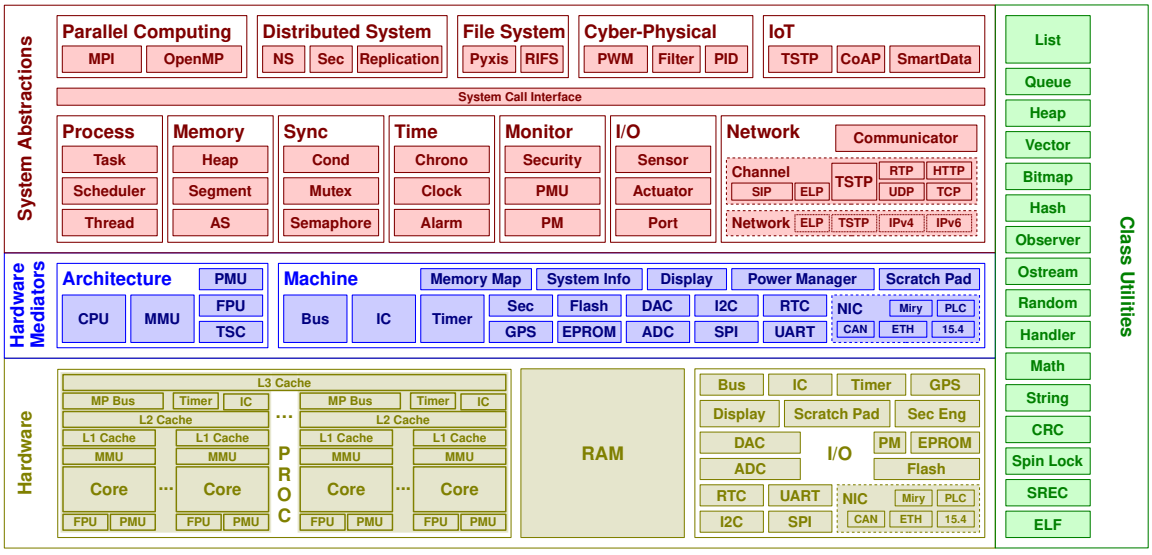


Figura 8 – Diagrama de Blocos da Arquitetura do EPOS

Este trabalho fundamenta-se nos princípios arquiteturais do EPOS, adotando o uso de abstrações de custo nulo e a especialização em tempo de compilação para nortear

o projeto da camada de software de baixo nível. No entanto, o escopo de aplicação difere: enquanto o EPOS foca na síntese adaptativa de SO e na co-síntese de hardware e software, o presente trabalho aplica essas diretrizes ao desenvolvimento de um hipervisor *bare-metal* para a arquitetura RISC-V. Assim, em vez de sintetizar um SO sob medida para uma aplicação específica, o objetivo é fornecer uma infraestrutura de virtualização enxuta e temporalmente previsível, capaz de isolar múltiplos sistemas operacionais e cargas de trabalho heterogêneas.

4.2 *BAO HYPERVISOR*

O *Bao* (MARTINS et al., 2020) é um hipervisor *bare-metal* e autossuficiente projetado para sistemas embarcados de criticidade mista. De modo análogo à solução proposta neste trabalho, o *Bao* opera de forma independente de um SO hospedeiro. No entanto, ele baseia-se em suporte dedicado de hardware, exigindo a extensão H da arquitetura RISC-V, além de empregar um particionamento estático rígido de recursos. Em contrapartida, o hipervisor desenvolvido nesta pesquisa viabiliza a virtualização mesmo sem a extensão H e introduz o suporte a *time-sharing*, permitindo o compartilhamento temporal de núcleos de processamento entre diferentes domínios.

4.3 MIRALIS

O Miralis (CASTES et al., 2024) adota uma abordagem alternativa ao empregar mecanismos computacionais semelhantes aos deste trabalho para estabelecer uma camada posicionada abaixo do firmware. A premissa central do Miralis fundamenta-se na constatação de que raramente os fabricantes disponibilizarão firmwares formalmente verificados para o modo de máquina. Para mitigar essa vulnerabilidade, o Miralis rebaixa a execução do firmware tradicional para o modo usuário, mantendo uma interface segura, padronizada e formalmente verificada no nível mais privilegiado. Para isso, utiliza puramente o mecanismo de *trap-and-emulate* a fim de interceptar e emular com segurança as instruções privilegiadas. Adicionalmente, o Miralis é capaz de emular a extensão de hypervisor via software de forma transparente, viabilizando a execução de hipervisores como o Bao (MARTINS et al., 2020) mesmo em plataformas desprovidas desse suporte em hardware.

4.4 HSP-V

O HSP-V (SOUSA et al., 2024) combina os domínios do OpenSBI com a PMP para prover ambientes de execução isolados em processadores RISC-V. De maneira semelhante

a este trabalho, o HSP-V permite a execução de múltiplos SOs em uma mesma plataforma de hardware, mesmo na ausência da extensão H.

Além do isolamento de memória, a pilha de *software* do HSP-V oferece mecanismos como o particionamento e injeção direta de interrupções, particionamento de *cache* e isolamento em nível de plataforma para dispositivos com suporte a DMA. Contudo, o HSP-V é classificado por seus criadores como *Hypervisor-Less*. Por essa razão, ele não implementa uma camada de emulação nem mecanismos para o compartilhamento ou multiplexação de periféricos, restringindo a arquitetura à atribuição estática e exclusiva de dispositivos entre os domínios.

5 PROJETO

Assim como o EPOS, este trabalho baseia-se no conceito de especialização de classes C++. O sistema é estruturado fundamentalmente em duas camadas principais: a camada de mediadores de hardware, composta por classes altamente especializadas e dedicadas à interação direta com os componentes físicos, e a camada de abstração, que utiliza os mediadores para implementar as funcionalidades do sistema. Adicionalmente, há a camada de virtualização, que atua como ponte entre os sistemas virtualizados, os mediadores e as próprias abstrações da plataforma.

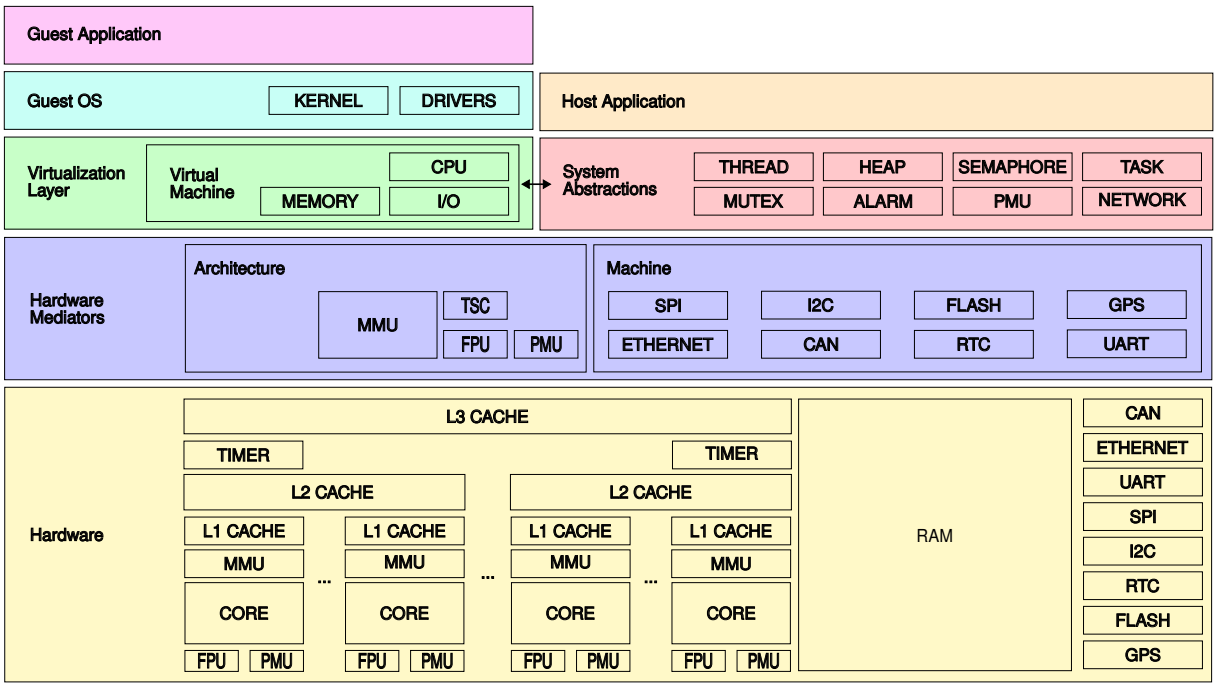


Figura 9 – Diagrama de Blocos das Classes do Projeto

Como ilustrado na Figura 9, o hipervisor desenvolvido neste trabalho — a exemplo de grande parte dos SO modernos — suporta dois tipos de aplicações: aquelas executadas sobre máquinas virtuais e aquelas mapeadas para rodar no próprio hipervisor sem a necessidade de uma VM. Essa abordagem é vantajosa em diversos cenários, como na execução de tarefas em que a latência deve ser mínima e o desempenho máximo, além de cenários focados em verificação e auditoria de segurança.

5.1 VMs

VMs são a correspondência em software — em nível de virtualização — de máquinas físicas, podendo conter CPUs virtuais, memória e dispositivos periféricos. Em correspondência com SOs modernos, VMs funcionam de forma análoga a processos no projeto. Por definição, a criação de uma VM deve receber uma porção de memória alocada para si; esta será a região delimitada e autorizada a ser acessada pelo convidado. Além disso, para cada núcleo esperado pelo convidado, a VM criará uma CPU virtual, ficando também responsável por criar e gerenciar os dispositivos virtuais.

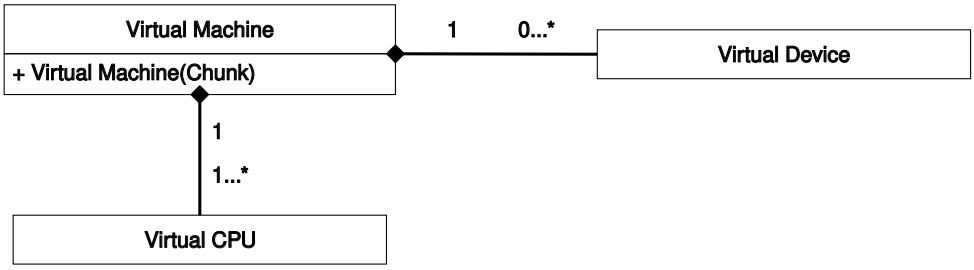


Figura 10 – Diagrama de Classes Simplificado de VMs

5.2 CPU VIRTUAL

SOs convidados não possuem conhecimento dos núcleos físicos do processador da plataforma. Por isso, grande parte dos hipervisores, incluindo este, implementa o conceito de CPUs virtuais, que simulam na prática um núcleo físico do processador. Dessa forma, para cada CPU de uma VM, cria-se uma *thread* encarregada de simular a execução de código do núcleo.

Além da execução de código, as CPUs virtuais também são responsáveis por redirecionar eventos às suas respectivas VMs. Como existe um mapeamento um para um entre *threads* e CPUs virtuais, quando ocorre um evento, é possível verificar o ponteiro da CPU virtual em execução naquele instante e, por meio dela, identificar a VM correspondente.

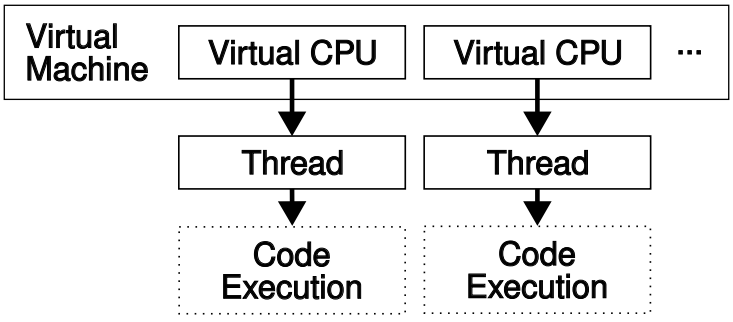


Figura 11 – Diagrama de Mapeamento de CPUs Virtuais Para Threads

5.3 DISPOSITIVOS VIRTUAIS

Dispositivos virtuais representam, para o SO convidado, dispositivos de hardware que podem ser utilizados para comunicação com a plataforma, como temporizadores e dispositivos de IO. Eles abstraem os mecanismos de hardware utilizados pelo hipervisor, permitindo que o convidado interaja com esses recursos sem acessar diretamente os dispositivos físicos.

5.3.1 Comunicação: Convidado para Hardware

A comunicação do SO convidado com o hardware é intermediada pelo hipervisor. As requisições do convidado podem chegar ao hipervisor tanto por meio da interceptação de operações de acesso ao hardware quanto por chamadas direcionadas diretamente aos dispositivos virtuais. Em ambos os casos, o hipervisor encaminha a requisição ao objeto da VM correspondente, que identifica o dispositivo virtual responsável pela operação. O dispositivo virtual, por sua vez, traduz e encaminha a requisição ao mediador de hardware correspondente, conforme ilustrado na Figura 12.

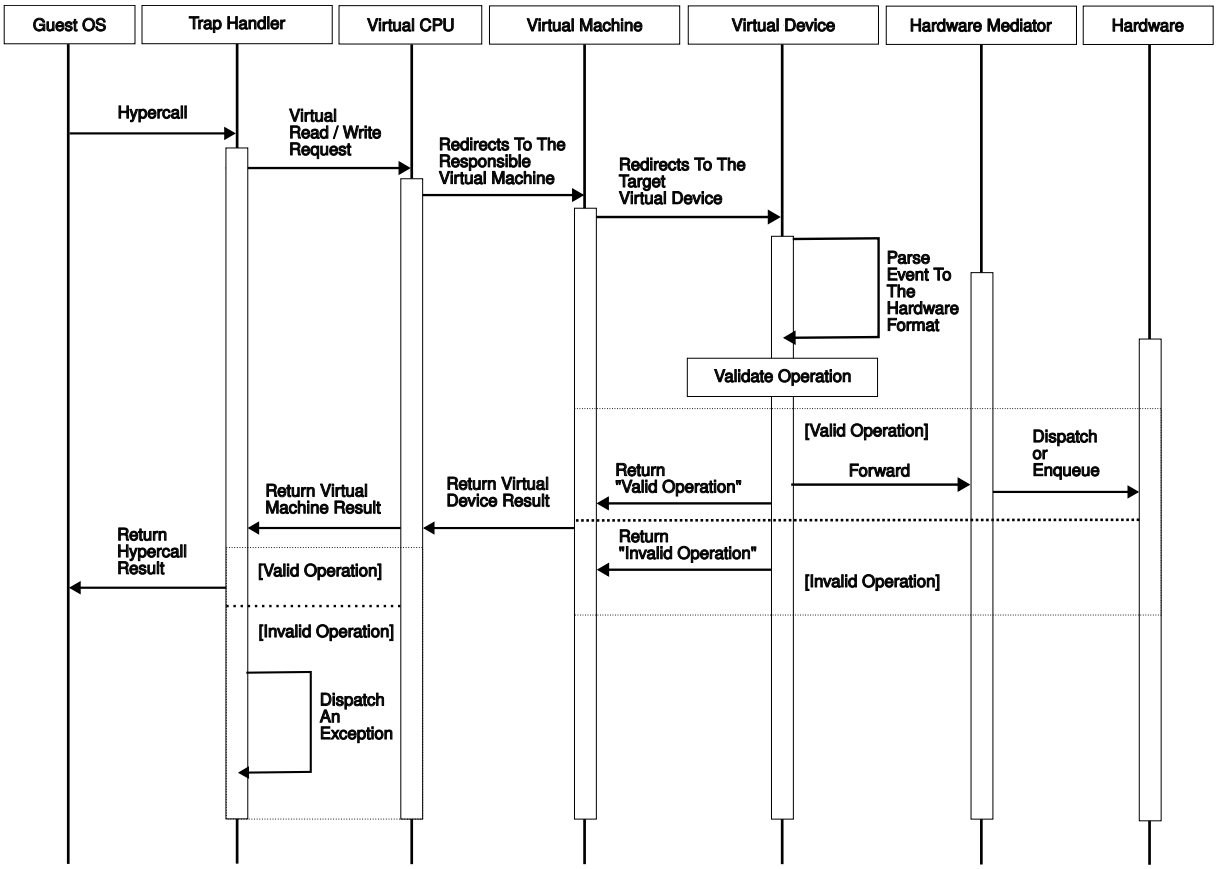


Figura 12 – Diagrama do Fluxo de Comunicação: Convidado para Hardware

5.3.2 Comunicação: Hardware para Convidado

O fluxo de notificação de um evento de hardware para o SO convidado é ilustrado na Figura 13. Esse mecanismo baseia-se no padrão de projeto *Observer*, no qual cada mediador de hardware atua como um elemento observável e cada dispositivo virtual funciona como um observador.

Quando ocorre um evento no mediador de hardware, este notifica seus observadores cadastrados, propagando a sinalização até que o dispositivo virtual correspondente processe a informação e a entregue ao domínio do SO convidado.

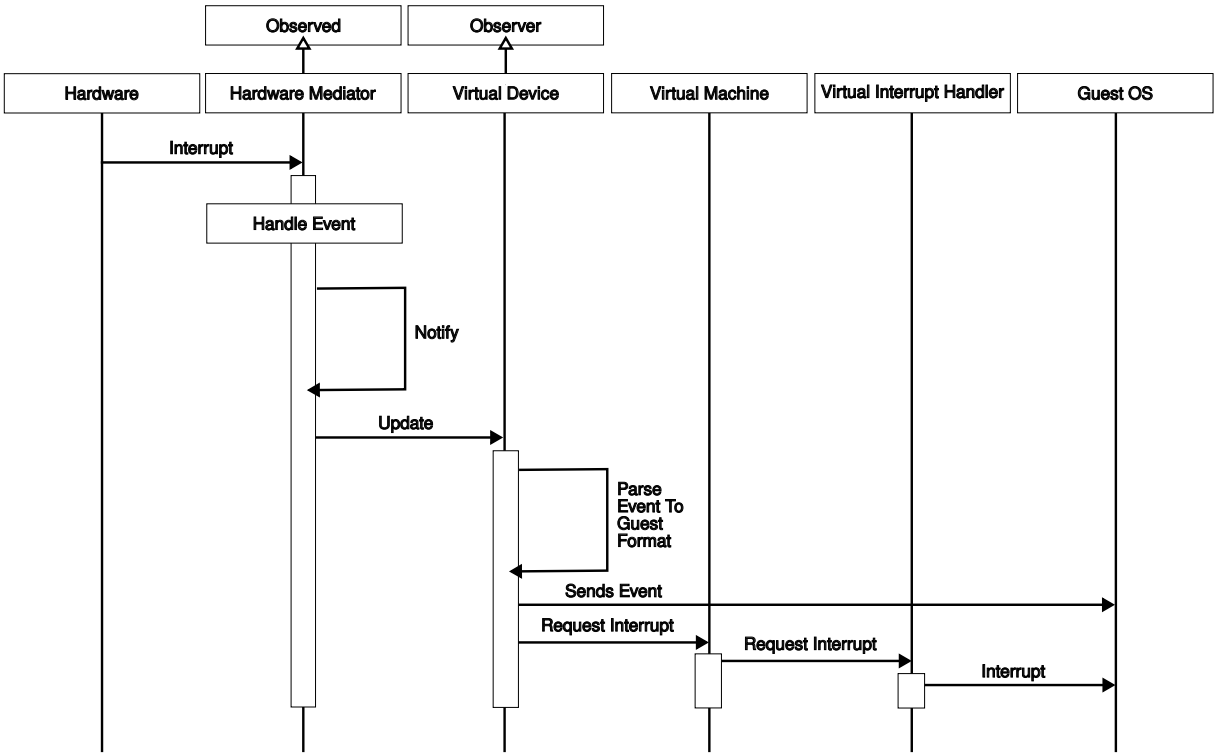


Figura 13 – Diagrama do Fluxo de Comunicação: Hardware para Convidado

5.4 INTEGRAÇÃO

A integração dos componentes, conforme mencionado anteriormente, ocorre por meio da agregação de todos os recursos de que uma VM necessita em sua própria classe. Além disso, cada mediador virtual possui um mapeamento específico para um mediador real, como ilustrado na Figura 14.

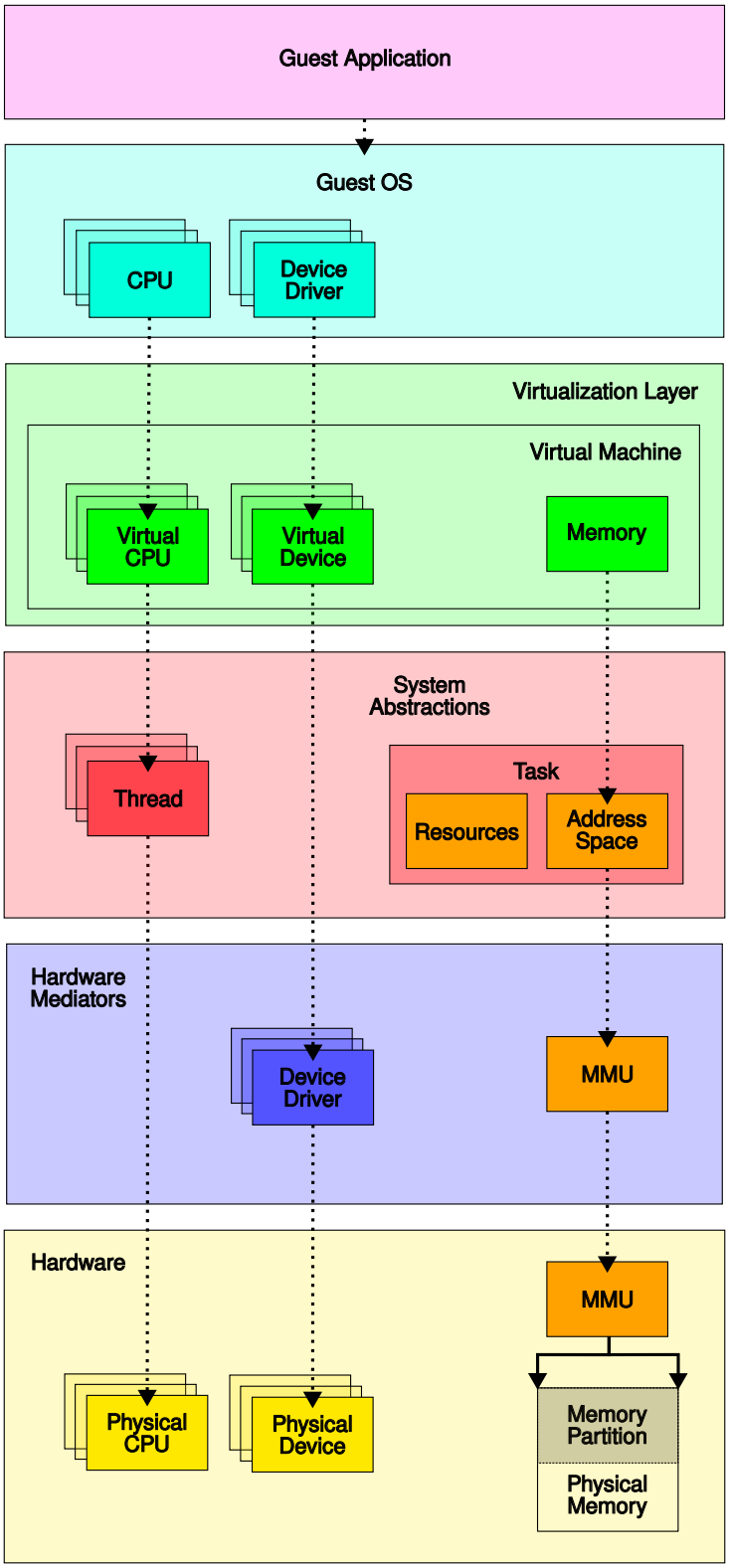


Figura 14 – Diagrama de Mapeamento de Componentes Virtuais para Reais

Formalizando essa estrutura, a interface mínima que a camada de virtualização fornece é apresentada na Figura 15.

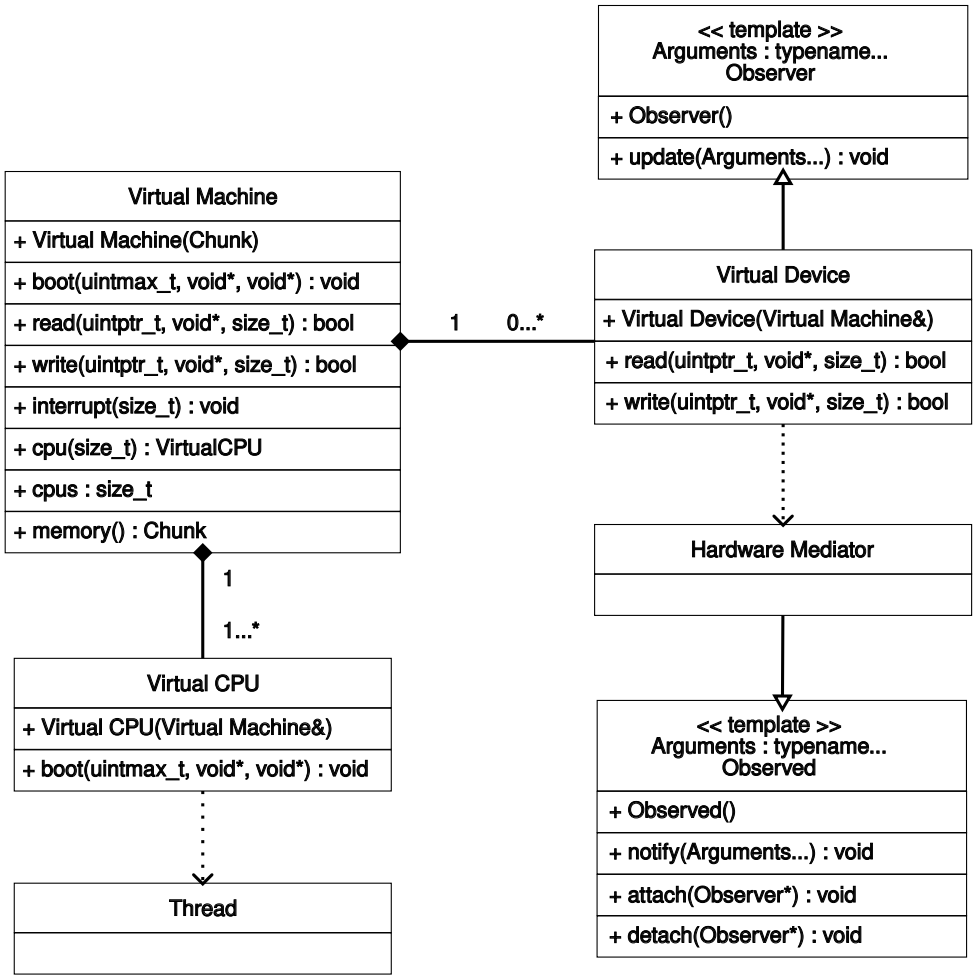


Figura 15 – Diagrama de Classes Relacionadas ao Processo de Virtualização

6 IMPLEMENTAÇÃO

6.1 VIRTUALIZAÇÃO BASEADA EM MODO *MACHINE*

Originalmente, o modo *Machine* da RISC-V foi projetado para ser o nível de privilégio mais elevado da arquitetura, sendo responsável pela execução de firmwares e pelo controle direto dos recursos de hardware. Sua função primária consiste na inicialização da plataforma e na configuração dos subsistemas fundamentais antes da transferência de controle para o SO hospedeiro.

Dada a natureza restritiva do acesso a determinados recursos de hardware — muitas vezes limitados exclusivamente a esse modo —, a arquitetura define camadas de abstração como a Supervisor Binary Interface (SBI). Essa interface atua como uma ponte padronizada, permitindo que níveis menos privilegiados solicitem serviços ao firmware residente no modo *Machine*.

No contexto deste trabalho, o modo *Machine* é utilizado de forma não convencional: em vez de empregá-lo apenas para prover a SBI, ele é utilizado como a base para a implementação de um hipervisor completo.

Essa abordagem explora o fato de que o modo *Machine* possui autoridade irrestrita sobre os demais níveis de privilégio, permitindo interceptar, controlar e gerenciar a execução dos sistemas convidados. Para viabilizar esse controle, o hipervisor emprega os recursos exclusivos desse modo, incluindo seu conjunto dedicado de CSRs, além de mecanismos próprios para tratamento de interrupções, proteção de memória e gerenciamento de temporização. Dessa forma, o modo *Machine* fornece a soberania necessária para emular por software funcionalidades que, em cenários convencionais, exigiriam suporte de hardware por meio de extensões de virtualização dedicadas.

6.2 VIRTUALIZAÇÃO DE TEMPORIZADOR

Por definição da arquitetura RISC-V, na ausência de extensões específicas de hardware, os recursos de temporizador — especificamente os registradores *mtime* e *mtimecmp* — são exclusivos e gerenciados pelo modo *Machine*.

Para contornar essa restrição em ambientes virtualizados, os SOs convidados — executados em modo S ou VS — utilizam a SBI para gerenciar eventos temporais. Dessa forma, não é necessária nenhuma adaptação estrutural no SO convidado, uma vez que a própria especificação padrão da arquitetura prevê o uso da SBI para essa finalidade.

6.2.1 Leitura do Temporizador

A leitura do tempo atual pelo convidado é feita por meio do CSR *time*, acessado comumente através da pseudo-instrução *rdtime*.

Quando um modo com menor privilégio tenta realizar essa leitura sem que o acesso direto tenha sido liberado, o processador dispara uma *Illegal Instruction*. O hipervisor intercepta essa exceção calcula o tempo virtualizado correspondente e devolve o valor correto ao registrador de destino do convidado.

6.2.2 Programação de Temporizador

Como o convidado não possui acesso direto de escrita ao registrador *mtimecmp*, o agendamento de interrupções é realizado por chamadas SBI — extensão *TIME*.

Quando o SO convidado precisa agendar um evento futuro, ele realiza uma chamada de sistema solicitando o registro do próximo limite temporal. O firmware recebe a requisição e ao atingir o tempo programado, injeta a interrupção sinalizando o bit *Supervisor Timer Interrupt Pending* (STIP) do registrador MIP da CPU virtual correspondente.

6.3 VIRTUALIZAÇÃO DE OCIOSIDADE

Algumas plataformas RISC-V possuem o bit *Timeout Wait* (TW) no registrador *mstatus*, dedicado a interceptar e restringir a execução da instrução *Wait For Interrupt* (WFI) exclusivamente ao modo *Machine*. Quando um SO convidado — em modo *Supervisor* — tenta executar essa instrução com o bit *TW* ativado, é gerada uma *Illegal Instruction*. Essa instrução é capturada e emulada pelo hipervisor, que coloca a VM em estado de repouso até o recebimento da próxima interrupção pertencente a ela.

Esse mecanismo é fundamental em cenários onde múltiplas VMs compartilham o mesmo núcleo. Sem a interceptação do estado de ociosidade, o SO convidado continuaria consumindo tempo de CPU desnecessariamente até ser interrompido pelo escalonador do hipervisor. Com a virtualização de ociosidade ativa, a transição para o estado de repouso é detectada imediatamente, permitindo que o processador seja cedido de forma eficiente a outra unidade de execução ativa.

6.4 VIRTUALIZAÇÃO DE DISPOSITIVOS

Dispositivos externos em sistemas RISC-V são, em sua maioria, acessados por meio de *Memory-Mapped I/O* (MMIO), no qual registradores de controle e dados dos dispositivos são mapeados em endereços do espaço de memória. Dessa forma, uma operação de leitura ou escrita em um determinado endereço pode representar uma operação sobre um

dispositivo, em vez de um acesso à memória convencional. A virtualização desses dispositivos pode, portanto, ser realizada interceptando as operações de memória destinadas às regiões correspondentes aos dispositivos virtuais e emulando seu comportamento.

Cada máquina virtual possui uma região de memória própria. Os acessos do convidado a essa região são permitidos pela PMP, enquanto as demais regiões de memória permanecem protegidas. Assim, quando o sistema convidado tenta acessar um endereço que não pertence à sua região de memória, o processador gera uma exceção de acesso, como *Load Access Fault* ou *Store Access Fault*, e transfere o controle para o hipervisor.

O hipervisor utiliza essas exceções para distinguir acessos inválidos à memória de acessos destinados a dispositivos virtuais. Quando o endereço acessado corresponde à região de MMIO de um dispositivo virtual, o hipervisor decodifica a instrução que originou a exceção e identifica o tipo da operação, o endereço acessado, seu tamanho e, no caso de uma escrita, o valor que deveria ser armazenado.

No caso de uma operação de leitura, o hipervisor solicita ao dispositivo virtual correspondente que realize a operação. O dispositivo virtual processa a requisição de acordo com seu estado interno e retorna o valor correspondente. O hipervisor então escreve esse valor no registrador de destino do contexto do convidado e ajusta o contador de programa para que a instrução que causou a exceção não seja executada novamente.

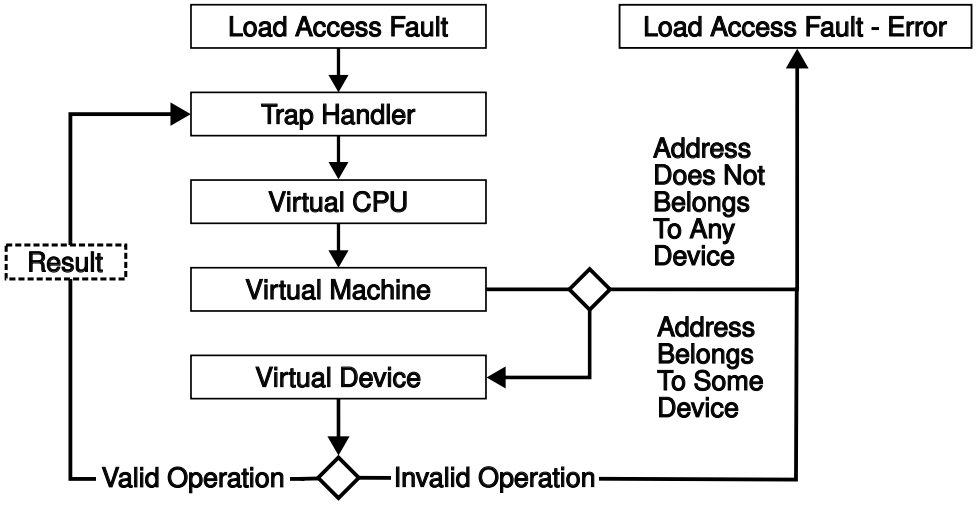


Figura 16 – Emulação de Dispositivos via *Load Access Fault*

Para operações de escrita, o processo é semelhante. O hipervisor identifica o valor que seria escrito pelo convidado e o encaminha ao dispositivo virtual correspondente. O dispositivo pode atualizar seu estado interno, produzir efeitos colaterais ou encaminhar a operação para o mediador de hardware correspondente.

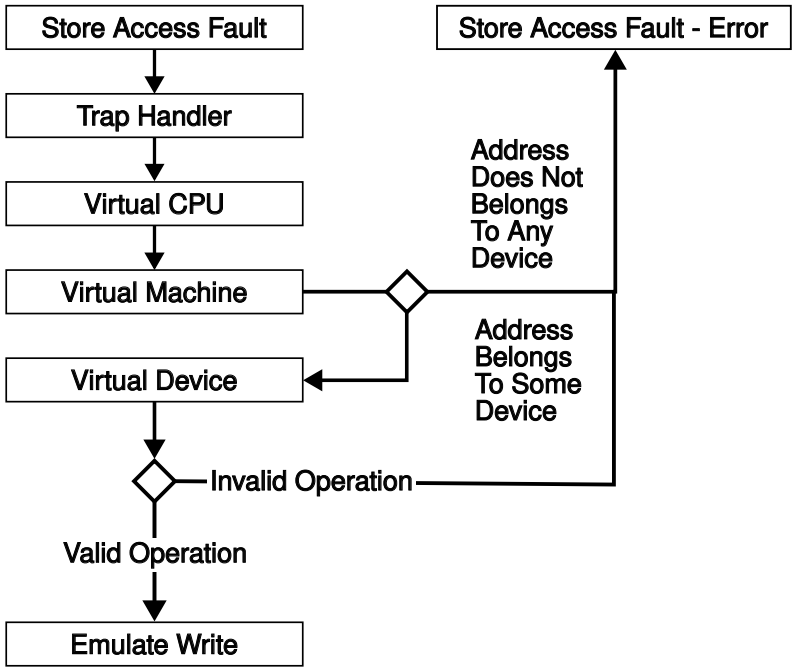


Figura 17 – Emulação de Dispositivos via *Store Access Fault*

Caso o endereço não corresponda a nenhum dispositivo virtual ou a operação solicitada não seja suportada pelo dispositivo, o acesso não pode ser emulado. Nesse caso, o hipervisor pode tratar a operação como um acesso inválido e encaminhar uma exceção ao sistema convidado ou, dependendo da política adotada, encerrar a VM.

6.5 VIRTUALIZAÇÃO DE INTERRUPÇÕES EXTERNAS

A virtualização de interrupções é realizada por meio de uma PLIC virtual, exposta ao SO convidado como um dispositivo mapeado em memória. Para o convidado, a PLIC é acessada da mesma forma que o mecanismo original de hardware, pois é virtualizada por meio da interceptação de instruções de memória descritas na seção anterior.

Quando dispositivos virtuais enviam requisições de interrupção para a PLIC virtual, esta atualiza seus estados internos para que o SO convidado possa identificar e ler a interrupção recebida. Em seguida, a PLIC virtual sinaliza a interrupção às CPUs virtuais pertencentes à mesma VM, definindo o bit SEIP como 1. O bit permanece ativo até que a interrupção seja tratada e a PLIC virtual o redefina para 0.

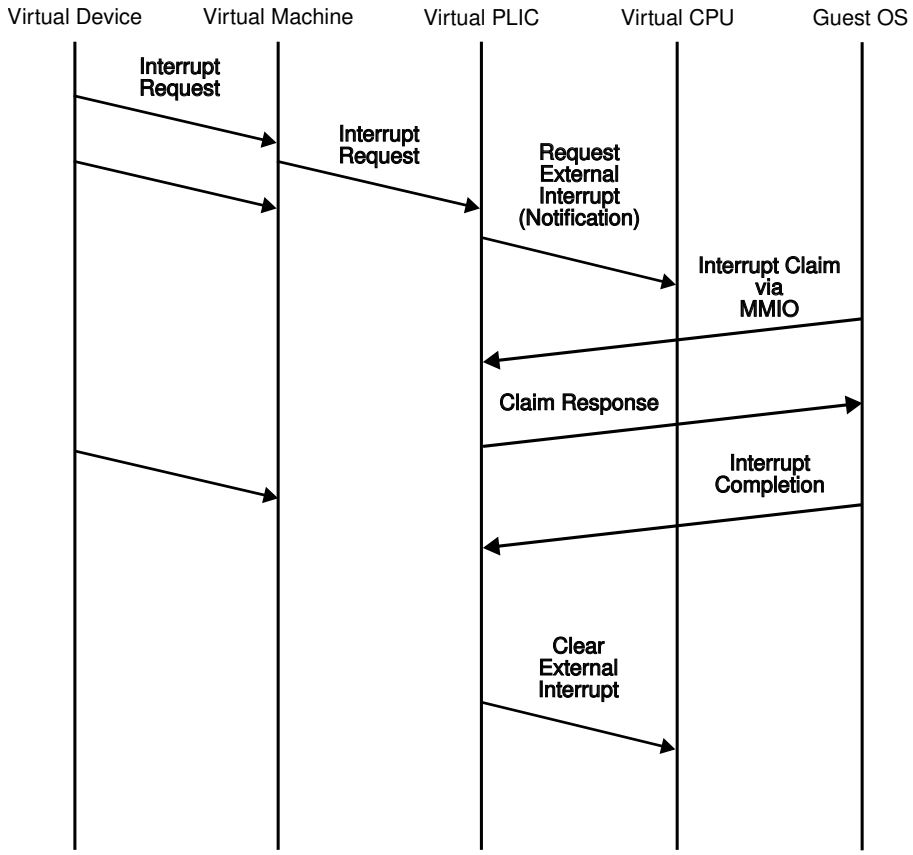


Figura 18 – Diagrama de Virtualização de Interrupções Externas na RISC-V

6.6 VIRTUALIZAÇÃO DA COMUNICAÇÃO ENTRE PROCESSADORES

A virtualização de interrupções de software na arquitetura RISC-V, um dos mecanismos utilizados para a comunicação entre processadores, baseia-se na intermediação das interrupções pelo hypervisor. Em uma implementação que não dispõe da extensão H, o acesso ao mecanismo de interrupção de software entre *harts* é realizado em modo *Machine*, por meio do registrador MSIP. Dessa forma, o SO convidado, executando em modo *Supervisor*, não possui acesso direto a esse registrador e, consequentemente, não pode gerar diretamente uma interrupção de software para outro processador físico. A geração e a entrega das interrupções são, portanto, intermediadas pelo hypervisor, que as associa às respectivas CPUs virtuais.

Nessa implementação, o SO convidado solicita o envio de uma interrupção por meio da interface de serviços definida pelo *OpenSBI*. Ao interceptar essa requisição, o hypervisor a direciona para a CPU virtual responsável pelo processamento da chamada. A CPU virtual identifica as CPUs virtuais de destino e encaminha a solicitação de interrupção. Quando a CPU virtual de destino está em execução, o hypervisor utiliza uma interrupção

de software física para sinalizar o *hart* correspondente. Após o processamento dessa interrupção, a CPU virtual de destino registra a interrupção como pendente e a entrega ao SO. Caso a CPU virtual de destino não esteja ativa, a interrupção é apenas marcada como pendente. Nesse caso, sua entrega é realizada quando a CPU virtual retomar a execução e seu estado for novamente atualizado pelo hypervisor.

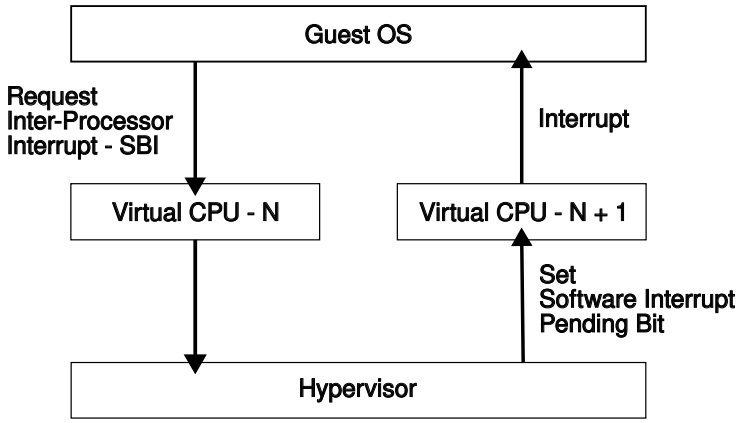


Figura 19 – Diagrama de Envio de uma IPI Virtual na RISC-V

6.7 ATUALIZAÇÃO DE ESTADO DAS CPUS VIRTUAIS

Em determinados momentos, o hipervisor pode optar por interferir de forma assíncrona na execução do convidado — analogamente à relação de interrupção entre o hardware e o SO —, injetando eventos na execução da CPU virtual. Para evitar problemas de concorrência e reduzir a *Trusted Computing Base* (TCB), apenas a própria CPU virtual tem permissão para alterar seu estado local. No entanto, qualquer componente do hipervisor pode sinalizar um evento como pendente, garantindo que ele seja lido e processado na próxima janela de atualização daquela CPU. Esses eventos são majoritariamente representados na forma de interrupções, contudo também podem consistir em alterações diretas de registradores ou até desligamento da VM.

As atualizações de estado ocorrem por duas vias principais: as síncronas, disparadas quando outro componente do sistema envia uma IPI de atualização para a CPU Virtual de destino; e as periódicas, executadas durante a ativação da CPU Virtual — seja por *boot* ou reescalonamento — e a cada *tick* do temporizador do sistema, caso ela esteja em execução.

6.8 ESCALONAMENTO DE CPUS VIRTUAIS

As CPUs virtuais, definidas pelo projeto e independentemente da arquitetura, possuem uma correspondência um para um com as *threads* do sistema. Da mesma forma

que as *threads* estão para os processos, as CPUs virtuais estão para uma VM. Com isso, o escalonamento das CPUs virtuais responde diretamente ao escalonamento da *thread* mapeada a ela. Ou seja, aplicam-se aqui todos os conceitos de priorização, temporização e organização interna do escalonador do sistema.

7 RESULTADOS

7.1 MATERIAIS

Todos os experimentos deste trabalho foram realizados na plataforma *VisionFive 2*. As principais características de hardware relevantes para o estudo são apresentadas a seguir:

- System on a Chip (SoC): StarFive JH7110;
- Memória DRAM: 8 GB LPDDR4;
- Interface Gigabit Ethernet: Synopsys DesignWare DWMAC 5.20;
- PMIC: X-Powers AXP15060.

O SoC StarFive JH7110 opera em frequências de até 1,5 GHz e conta com uma arquitetura heterogênea composta por seis núcleos de processamento:

- 4 núcleos U74 (64 bits), compatíveis com a ISA RISC-V RV64GC;
- 1 núcleo S7 (64 bits), compatível com a ISA RISC-V RV64IMAC;
- 1 núcleo E24 (32 bits), compatível com a ISA RISC-V RV32IMAFCB.

Apesar da heterogeneidade do SoC, as avaliações restringiram-se aos quatro núcleos U74. Os núcleos S7 e E24 foram desconsiderados por não suportarem o modo supervisor, requisito indispensável para a execução dos mecanismos de virtualização analisados.

7.2 AVALIAÇÃO DE DESEMPENHO: NATIVO VS. VIRTUALIZADO

Com o intuito de avaliar a propriedade de eficiência de virtualização, foram realizados testes experimentais para mensurar o impacto do hipervisor no desempenho das aplicações executadas sobre as VMs. Para essa avaliação, foram utilizados 10 *benchmarks* da suíte *MiBench* (GUTHAUS et al., 2001):

- ***Basicmath*:** realiza cálculos matemáticos básicos, avaliando o desempenho em processamento numérico e operações aritméticas;

- ***Dijkstra*:** aplica o algoritmo de Dijkstra para calcular os caminhos mais curtos em um grafo, simulando cargas de trabalho de roteamento de redes e algoritmos de navegação;
- ***Qsort*:** executa a ordenação de um arranjo por meio do algoritmo *QuickSort*;
- ***SHA*:** executa o *Secure Hash Algorithm*, mensurando o desempenho do sistema sob processamento intensivo de operações bit a bit;
- ***Susan*:** realiza o processamento de imagens para detecção de bordas, cantos e filtragem de ruído, representando tarefas de visão computacional e manipulação de matrizes de pixels.

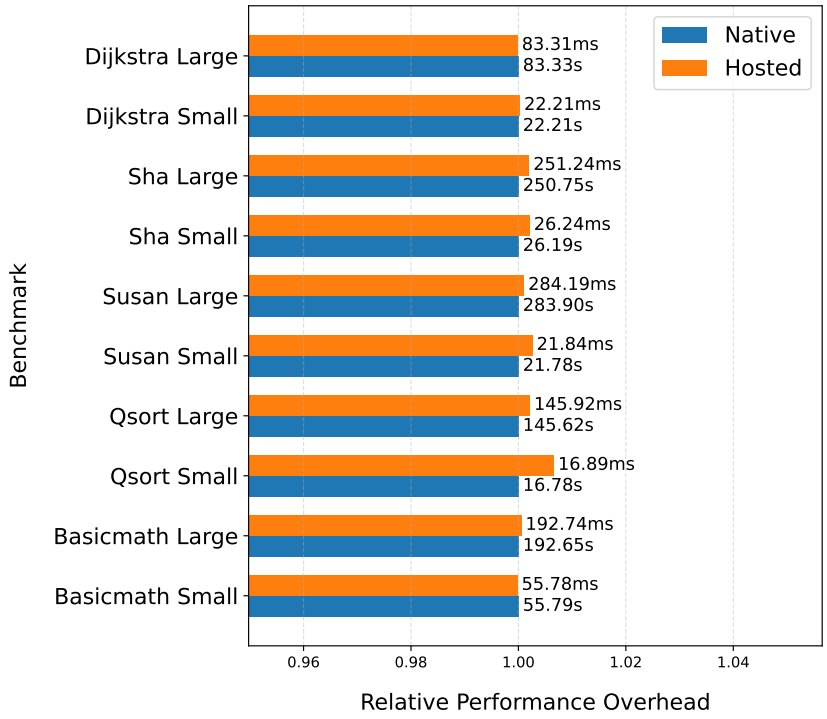


Figura 20 – Avaliação Experimental do *Overhead* de Virtualização com a Suíte MiBench.

As variações *Large* e *Small* da Figura 20 diferem estritamente no volume dos dados de entrada fornecidos a cada algoritmo, permitindo analisar o comportamento do hipervisor tanto sob condições de estresse prolongado quanto em execuções de curta duração.

Os resultados experimentais confirmam a eficiência do hipervisor, apresentando um *overhead* máximo inferior a 2% em todos os testes. Tal fato, evidencia que a camada de virtualização mantém o desempenho das VMs próximo ao da execução nativa.

7.3 AVALIAÇÃO DE DETERMINISMO E ISOLAMENTO

Para avaliar e validar o determinismo desse projeto, foi modelado um cenário onde o hipervisor seria utilizado em uma *Electronic Control Unit* (ECU) veícular. Para isso foram empregados quatro RTOS EPOS e um Linux rodando uma atividades intensivas para tentar ao maximo gerar intrusividade.

As quatro instancias do EPOS rodam, cada uma, uma atividade diferente: Um dos EPOS roda um kalman filter em 5′000 amostras, um roda uma convlução de um kernel 3×3 em uma imagem 512×512, um roda transformada rápida de fourrier, e o outro roda conversão de coordenadas esféricas para coordenadas cartesianas em 50′000 pontos.

Três das tarefas, com exeção do kalman filter, possuem uso intensivo de memória, em um cenário ideal, o objetivo desse experimento é observar uma variação nula ou mínima nesse, e uma maior nas outras uma vez que a principal forma de interferencia é via cache de memória.

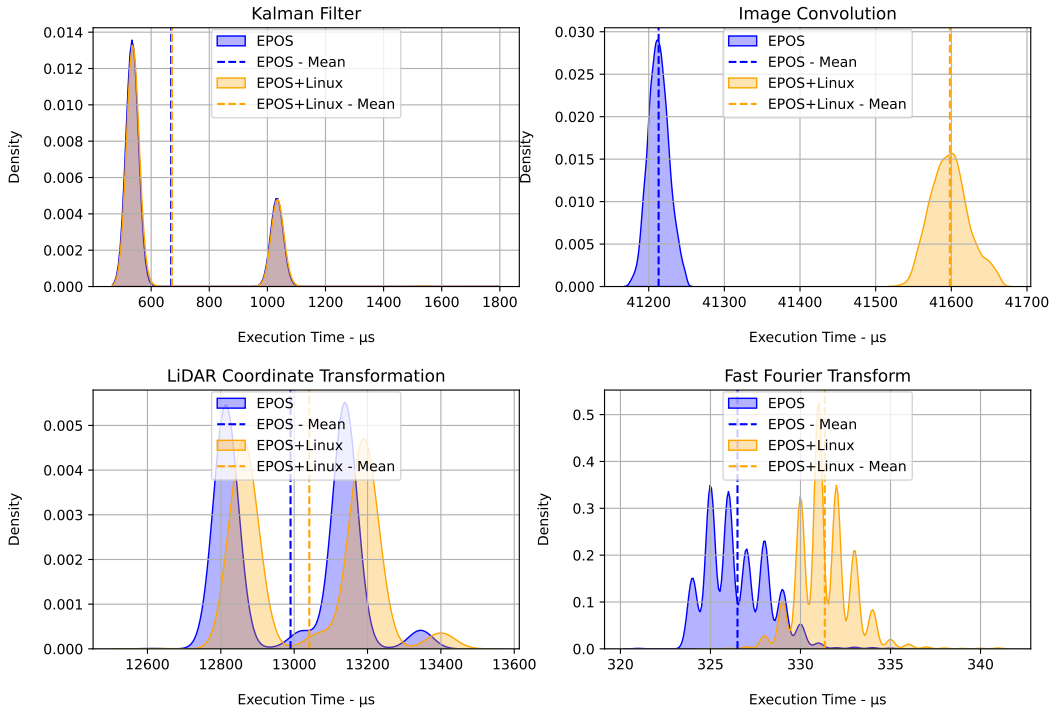


Figura 21 – Distribuição de Densidade dos Tempos de Execução das Tarefas do Cenário Veicular.

A Figura 21 apresenta as distribuições de densidade probabilística para cada uma das instâncias do RTOS, avaliadas em isolamento e sob interferência do Linux em segundo plano. Complementarmente, a Tabela 1 sintetiza a média e o desvio padrão obtidos nesses experimentos.

Tarefa	**EPOS** (μ)	**EPOS** (σ)	**EPOS+Linux** (μ)	**EPOS+Linux** (σ)
Kalman Filter	668,56	221,86	671,50	222,37
Convlução	41.212,63	17,37	41.597,71	29,89
Coordenadas	12.990,10	171,64	13.041,25	174,57
FFT	326,58	2,15	332,25	4,75

Tabela 1 – Comparação dos Tempos de Execução nos Cenários com e sem a Intrusividade do Linux.

Como esperado, os resultados indicam que as tarefas com uso mais intensivo de memória foram as mais afetadas. Ainda assim, a interferência máxima manteve-se abaixo de 2%.

Para comprovar que a maior parte da interferência provém da memória, os dois cenários foram reavaliados com a ativação dos mecanismos de particionamento de cache da plataforma utilizada. Esse mecanismo divide a cache em partes iguais para cada núcleo da CPU, amenizando a interferência mútua.

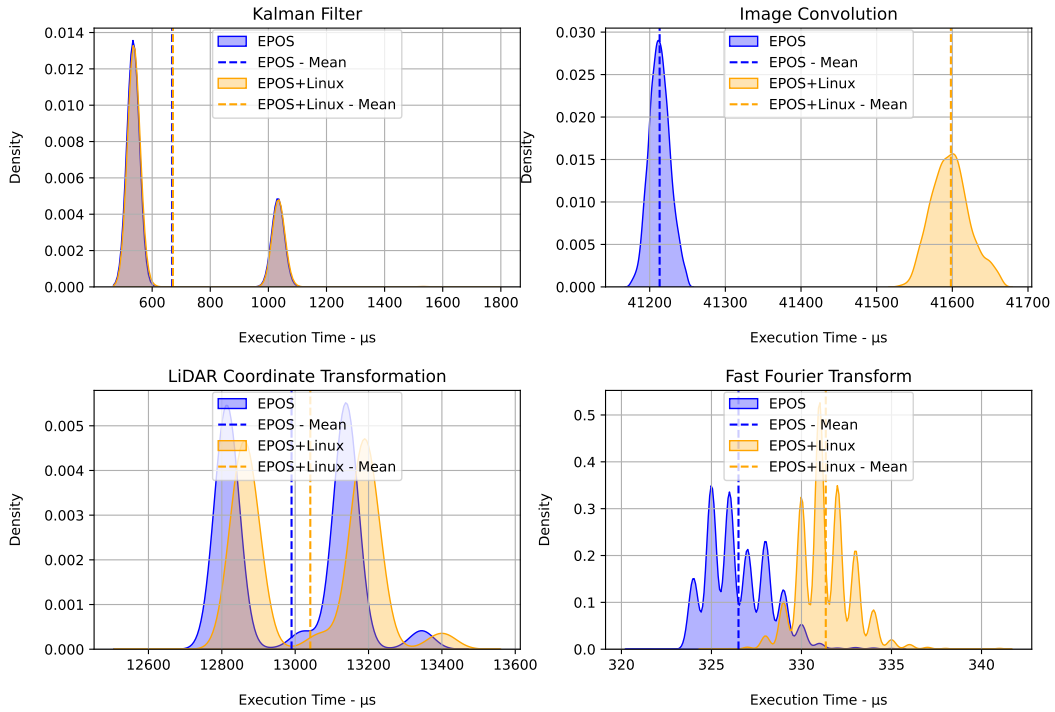


Figura 22 – Distribuição de Densidade dos Tempos de Execução das Tarefas do Cenário Veicular com Particionamento da Cache.

O particionamento de cache reduz a interferência máxima para menos de 1,5%, embora ainda persista uma parcela residual decorrente da competição no barramento de

Tarefa	**EPOS** (μ)	**EPOS** (σ)	**EPOS+Linux** (μ)	**EPOS+Linux** (σ)
Kalman Filter	686,04	230,87	688,65	230,92
Convlução	44 630,28	39,79	45 005,51	50,90
Coordenadas	12 844,28	57,48	12 898,07	58,02
FFT	327,10	2,51	331,41	2,55

Tabela 2 – Comparação dos Tempos de Execução nos Cenários com e sem a Intrusividade do Linux com Particionamento da Cache.

memória.

8 CONCLUSÕES

Este trabalho apresentou o projeto, a implementação em C++ e a avaliação experimental de um hipervisor Tipo-1 de tempo real voltado para sistemas embarcados baseados na arquitetura RISC-V. A solução desenvolvida responde aos desafios impostos pela computação de criticidade mista, viabilizando a coexistência e o isolamento seguro entre cargas de trabalho determinísticas e GPOS, como o Linux, sobre uma mesma plataforma de hardware.

Uma das principais contribuições desta pesquisa consiste na superação da dependência de extensões de hardware dedicadas à virtualização. Diante da escassez da extensão H em diversos processadores embarcados RISC-V de menor custo, a infraestrutura proposta utilizou de forma não convencional o modo *Machine* como camada soberana de execução, sintetizando via *software* os mecanismos de interceptação, emulação, proteção de memória e gerenciamento de máquinas virtuais. Adotando os princípios arquiteturais do EPOS, o projeto empregou de forma intensiva metaprogramação em C++ e especialização em tempo de compilação, alcançando abstrações de custo nulo e reduzindo a sobrecarga em tempo de execução.

Os resultados obtidos na plataforma experimental VisionFive 2 comprovaram a eficiência, a previsibilidade temporal e o grau de isolamento providos pelo hipervisor:

- **Baixo Overhead Computacional:** A avaliação com dez *benchmarks* da suíte MiBench demonstrou que a camada de virtualização impõe um *overhead* inferior a 2% em relação à execução nativa, assegurando desempenho próximo ao do *hardware* real.

- **Isolamento Temporal e Determinismo:** No experimento focado em um cenário veicular de criticidade mista — composto por quatro instâncias do RTOS EPOS executando sob forte interferência do Linux em segundo plano —, a variação observada nos tempos de execução das tarefas críticas manteve-se abaixo de 2%.

- **Mitigação de Contagência em Memória:** Ao aplicar o mecanismo de particionamento da *cache* L2 disponibilizado pela plataforma, a interferência máxima entre os domínios foi reduzida para menos de 1,5%, confirmando que a interferência residual decorre precipuamente da competição pelo barramento de memória e que a previsibilidade das tarefas de tempo real é preservada.

Em síntese, o trabalho demonstra que é viável atingir alto desempenho, determinismo temporal e isolamento estrito em arquiteturas RISC-V sem suporte nativo à

virtualização em hardware, fornecendo uma base sólida para a consolidação de sistemas embarcados críticos de recursos limitados.

Além dos resultados técnicos alcançados, a realização deste projeto de Iniciação Científica (PIBIC) desempenhou um papel divisor de águas na minha formação acadêmica e profissional. A pesquisa permitiu conectar conceitos teóricos de arquitetura de computadores, sistemas operacionais e programação em baixo nível à prática da engenharia de software, enfrentando desafios reais no ecossistema RISC-V.

REFERÊNCIAS

CASTES, C. et al. Kicking the firmware out of the tcb with the miralis virtual firmware monitor. In: *Proceedings of the 2nd Workshop on Kernel Isolation, Safety and Verification*. New York, NY, USA: Association for Computing Machinery, 2024. (KISV '24), p. 8–15. ISBN 9798400713019. Disponível em: ⟨https://doi.org/10.1145/3698576.3698764⟩.

FRöHLICH, A.; SCHRöEDER-PREIKSCHAT, W. High performance application-oriented operating systems - the epos approach. In: *Anais do XI Symposium on Computer Architecture and High Performance Computing*. Porto Alegre, RS, Brasil: SBC, 1999. p. 3–9. ISSN 0000-0000. Disponível em: ⟨https://sol.sbc.org.br/index.php/sbac-pad/article/view/19765⟩.

GOLDBERG, R. P. Architectural principles for virtual computer systems. In: . [s.n.], 1973. Disponível em: ⟨https://apps.dtic.mil/sti/tr/pdf/AD0772809.pdf⟩.

GOLDBERG, R. P. Architecture of virtual machines. In: *Proceedings of the Workshop on Virtual Computer Systems*. New York, NY, USA: Association for Computing Machinery, 1973. p. 74–112. ISBN 9781450374279. Disponível em: ⟨https://doi.org/10.1145/800122.803950⟩.

GUTHAUS, M. et al. Mibench: A free, commercially representative embedded benchmark suite. In: *Proceedings of the Fourth Annual IEEE International Workshop on Workload Characterization. WWC-4 (Cat. No.01EX538)*. [S.l.: s.n.], 2001. p. 3–14.

INTERNATIONAL, R.-V. *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. 2024. ⟨https://docs.riscv.org/reference/isa/_attachments/riscv-unprivileged.pdf⟩.

MARTINS, J. et al. Bao: A Lightweight Static Partitioning Hypervisor for Modern Multi-Core Embedded Systems. In: BERTOGNA, M.; TERRANEO, F. (Ed.). *Workshop on Next Generation Real-Time Embedded Systems (NG-RES 2020)*. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020. (Open Access Series in Informatics (OASIcs), v. 77), p. 3:1–3:14. ISBN 978-3-95977-136-8. ISSN 2190-6807. Disponível em: ⟨https://drops.dagstuhl.de/entities/document/10.4230/OASIcs.NG-RES.2020.3⟩.

POPEK, G. J.; GOLDBERG, R. P. Formal requirements for virtualizable third generation architectures. *Commun. ACM*, Association for Computing Machinery, New York, NY, USA, v. 17, n. 7, p. 412–421, jul. 1974. ISSN 0001-0782. Disponível em: ⟨https://doi.org/10.1145/361011.361073⟩.

RUSSELL, R. virtio: towards a de-facto standard for virtual i/o devices. *SIGOPS Oper. Syst. Rev.*, Association for Computing Machinery, New York, NY, USA, v. 42, n. 5, p. 95–103, jul. 2008. ISSN 0163-5980. Disponível em: ⟨https://doi.org/10.1145/1400097.1400108⟩.

SOUSA, J. et al. Hsp-v: Hypervisor-less static partitioning for risc-v cots platforms. *IEEE Access*, v. 12, p. 71131–71144, 2024.

STANKOVIC, J. A. Misconceptions about real-time computing: A serious problem for next-generation systems. *Computer*, IEEE Computer Society Press, Washington, DC, USA, v. 21, n. 10, p. 10–19, out. 1988. ISSN 0018-9162. Disponível em: ⟨https://doi.org/10.1109/2.7053⟩.